Semantically Driven Table Augmentation System for Data Lakes

Undergraduate thesis - English translation

Abstract (Translated from the Chinese Abstract)

In recent years, the rapid development of data lake technologies has greatly improved the availability of data resources, opening new avenues for automatically discovering and integrating semantically related external tables. This provides innovative approaches to table augmentation and new opportunities to improve the performance of downstream machine learning tasks. However, existing table augmentation methods mainly rely on textual syntactic similarity and column-name similarity for matching. They still face major challenges when handling the semantic and structural heterogeneity common in data lakes. First, excessive reliance on textual similarity for table discovery makes it difficult to uncover implicit semantic relationships between tables. Second, reliance on column names or column-text similarity during augmentation can result in incorrect matches or large numbers of missing values.

To address these challenges, this graduation project develops a semantically driven table augmentation system for data lakes. Built with the FastAPI backend framework, the Vue3 frontend framework, pretrained language models, and large language models (LLMs), the system automates the complete workflow of table preprocessing, related table discovery, and table augmentation. Its workflow has four core steps. First, an embedding model converts table data into vector representations. Second, a multilayer HNSW index is constructed from these representations to enable efficient retrieval of similar tables. Third, an LLM analyzes the retrieved top-k related tables and generates augmentation suggestions. Finally, the system follows these suggestions to supplement data and impute missing values. The resulting optimized, augmented table is provided for downstream tasks.

The system is evaluated on the Santos benchmark and WWk datasets and compared with baseline methods including D3L, BM25, and TF-IDF in terms of precision, recall, and query latency. The experimental results show an approximately 10% improvement in accuracy over the strongest baseline, BM25, and an approximately 60% improvement in retrieval speed. In terms of data augmentation effectiveness, the augmented tables improve downstream task performance by approximately 40%, demonstrating the advantages of the proposed method in both related table discovery and data augmentation.

Keywords: data lake; table augmentation; semantics-driven; pretrained language model; large language model

Abstract (English Abstract in the Original)

In recent years, the rapid development of data lake technologies has greatly improved data accessibility, enabling new approaches for automatically discovering and integrating semantically related external tables. This opens up innovative opportunities for table augmentation and enhancing downstream machine learning tasks. However, current methods largely rely on textual and column name similarity, which struggle with semantic and structural heterogeneity common in data lakes, leading to issues such as failure to capture implicit semantic links and high rates of mismatches or missing values.

To address these challenges, this thesis presents a semantic-based table augmentation system for data lakes. The system is implemented using the FastAPI backend framework, the Vue3 frontend framework, and incorporates pretrained language models and large language models. It supports a fully automated pipeline encompassing table preprocessing, related table discovery, and augmentation. The system follows a four-step core workflow: First, it uses embedding models to convert table data into vector representations. Based on these vectors, a multi-layer HNSW index is constructed to enable efficient retrieval of similar tables. Next, an LLM analyzes the top-k retrieved candidate tables and generates augmentation suggestions. Finally, the system applies these suggestions to supplement data and fill in missing values. After completion, the system outputs an optimized and augmented table for use in downstream tasks.

System performance is evaluated using the Santos benchmark dataset and the WWk dataset, comparing against baseline methods such as D3L, BM25, and TF-IDF across three metrics: precision, recall, and query latency. Experimental results show that the proposed system improves precision by approximately 10% over the best-performing baseline BM25 and increases retrieval speed by about 60%. In terms of augmentation effectiveness, the enhanced tables yield around a 40% performance improvement in downstream tasks, demonstrating the system's significant advantages in both related table discovery and data augmentation.

KEY WORDS: Data Lake, Table Enhancement, Semantics-driven, Pretrained Language Model, Large Language Model

Contents

Abstract (translated)	4
Original English abstract	5
Contents	6
Chapter 1 Introduction	7
1.1 Background and Significance	7
1.2 Current Research	7
1.3 Work Presented in This Thesis	13
1.4 Organization of the Thesis	14
Chapter 2 Related Technologies	15
2.1 Embedding Model	15
2.2 Transformer Models and Contrastive Learning	16
2.3 BERT Model	16
2.4 Chapter Summary	17
Chapter 3 Detailed System Design and Implementation	18
3.1 Requirements Analysis	18
3.2 Architecture Design	20
3.3 Workflow Design	21
3.4 Chapter Summary	36
Chapter 4 System Testing and Analysis of Results	37
4.1 Test Objectives and Configuration	37
4.2 System Testing and Analysis	38
4.3 Chapter Summary	49
Chapter 5 Conclusions and Future Work	51
5.1 Summary of the Work	51
5.2 Future Work	51
References	53

Chapter 1 Introduction

1.1 Background and Significance

In recent years, with the rapid development of data lake technologies, more organizations and institutions have actively contributed to open-source datasets, significantly improving the availability of data resources. However, the massive volumes of data in data lakes are usually dispersed and lack unified management. Efficiently integrating and using these diverse data to support data science tasks remains a major challenge. A single table often provides insufficient information, limiting the effectiveness of data analysis and mining. During table augmentation, users frequently face a dilemma: either switch repeatedly between fragmented tools to build complex workflows, or spend substantial time on repetitive manual operations. Developing a semantically driven table augmentation system for data lakes that automatically discovers and integrates related tables to improve data science performance therefore has substantial theoretical and practical significance.

Researchers have proposed various solutions to related table discovery and integration. Traditional methods evaluate table relatedness mainly through term-frequency similarity, but this approach has clear limitations and cannot accurately capture complex semantic relationships between tables. Traditional methods are also computationally inefficient, resulting in lengthy retrieval processes. Recent advances in LLM technology offer new approaches to table discovery and integration. With their strong semantic understanding capabilities, LLMs can interpret table content more deeply. These advantages make it valuable to develop a new table augmentation system incorporating LLMs for more efficient and accurate discovery and augmentation.

This thesis designs and implements a semantically driven table augmentation system for data lakes. It rapidly discovers related tables and performs effective augmentation, allowing better use of tabular data in data lakes. Automated discovery and integration substantially reduce the time data scientists spend preparing data. By enriching the original table, the system can also improve a range of downstream data science tasks. In addition, it simplifies the augmentation process and provides a convenient, efficient data processing tool. Its straightforward, user-friendly interface lowers the technical barrier and enables more users to use table augmentation easily.

1.2 Current Research

Table augmentation involves two key stages: finding related tables during discovery and resolving conflicts during augmentation. Accordingly, this review focuses on two related research areas: related table discovery and conflict resolution in augmentation.

1.2.1 Data Lakes and the Related Table Discovery Task

A data lake is a system architecture for storing large volumes of raw data. It accommodates structured, semistructured, and unstructured data. A data lake does not require a predefined schema or structure; instead, data may be stored directly in their original form. This approach provides considerable flexibility and accommodates diverse data types and formats, supporting a wide range of analysis and processing needs.

Data lakes have four main advantages over traditional databases and data warehouses. First, they are highly flexible and accommodate different sources and formats to meet diverse analytical needs. Second, they are cost-effective, using low-cost storage solutions particularly suitable for massive datasets. Third, they democratize data by allowing more members of an organization to access data resources. Finally, they support advanced analysis, including machine learning and data mining, providing a rich data foundation for data scientists and analysts. These characteristics make data lakes important infrastructure for modern data analysis.

In a data lake, related tables are tables that have semantic or structural relationships with a user's original input table and can be integrated through UNION or JOIN operations [2]. Unionable tables generally have structures similar to the original table and can be combined row-wise to expand the amount of data. Joinable tables are generally semantically related to the original table and can be joined using common fields to add columns and enrich its information [4].

The related table discovery task identifies tables associated with a query table in terms of semantic relevance, structural compatibility, and data complementarity [2]. Semantic relevance means that the related and original tables are closely connected or highly similar in meaning. Structural compatibility means that they have similar structures, enabling UNION or JOIN operations to produce an augmented table. Data complementarity means that related tables supply additional information, enrich the original table's features, and improve downstream data science performance.

1.2.2 Related Table Discovery

In data lake table augmentation, related table discovery can be formally defined as retrieving the top-k tables most relevant to a source table Q from a data lake containing N tables. The most direct approach is a linear scan: traverse every table in the data lake, compute its similarity to the source table, and select the k tables with the highest scores. In practice, however, this approach is inefficient, with excessive computation and long execution times. Researchers have therefore proposed optimizations around two goals: improving query efficiency and improving discovery accuracy and recall. Existing approaches can be classified by their core techniques as follows.

1.2.2.1 Accurate Discovery of Related Tables

To discover related tables accurately, researchers focus on quantifying table relatedness more precisely. Traditional methods generally rely on simple textual matches, such as table names and column names, and struggle to capture deeper semantic information. Recent approaches based on relational graphs and machine learning improve semantic recognition and provide a stronger basis for related table discovery.

Column semantics and semantic relationships between different columns can represent table semantics. Khatiwada et al. [1] assign semantic annotations to each column and relational semantics to each column pair. These relations represent binary relationships between column values, improving feature extraction and the accuracy of table union search in data lakes. Existing and synthetic knowledge bases are used to identify relationships between columns and construct a semantic graph for each table, where nodes represent columns and edges represent relationships. Matching scores for column and relational semantics are then compared to obtain an overall union-search score and determine table similarity.

Bogatu et al. [2] propose D3L (Dataset Discovery in Data Lakes). They build hash-based indexes from dataset features, map table features into a common distance space, and define feature similarity as a measure of table relatedness. D3L determines table similarity using five features: attribute-name similarity, attribute-value overlap, word-embedding similarity, format-representation similarity, and domain-distribution similarity. These features account for multiple factors affecting similarity; adjusting their relative weights can improve semantic recognition and discovery accuracy. Similarly, Nargesian et al. [3] evaluate whether two attributes come from the same domain using set, semantic, and natural-language domains, thereby assessing table relatedness. Their integrated approach automatically chooses the most appropriate unionability assessment for each attribute pair, obtaining its highest similarity and capturing table similarity more intelligently.

Mixed-integer programming (MIP) is a mathematical optimization method for problems whose objective and constraints contain both continuous and integer variables. Chen et al. [4] formulate an optimization problem that uses MIP to select the best set of related tables. Beyond semantics, their approach considers table compatibility: whether tables can be joined through columns. The compatibility score combines column relevance and the likelihood of satisfying foreign-key constraints.

Hash values can represent the features of individual columns while reducing storage requirements. Santos et al. [7] propose related table discovery based on Quadrant Count Ratio (QCR) hashing.

QCR hashing divides the numerical plane into four quadrants and generates hash values so that tables with identical column names and numerical values in the same quadrant can match. Query-table hash entries are then generated, and tables sharing the largest number of entries are retrieved. Harmouch et al. [9] propose a novel hash method, Xash, that encodes the syntactic features of all column values to understand table semantics more deeply and discover related tables more accurately.

Converting table text into vectors captures semantic relationships, accounts for contextual meaning, and maps different data types into a shared vector space. Dong et al. [8] propose PEXESO, which converts every table column into a high-dimensional vector and evaluates table similarity through vector similarity. Fan et al. [14] further develop Starmie, training a column encoder through contrastive learning to produce contextualized column vectors. Each vector learns both its own column's semantic features and its relationships with other columns. Considering these relationships reduces incorrect matches and improves discovery accuracy.

Although these methods improve accuracy, they have limitations. Methods using intercolumn semantic relationships depend on knowledge bases and require complex semantic-graph construction. D3L integrates multiple features, but feature-similarity computation is time-consuming and requires parameter tuning. Nargesian et al.'s automatic assessment selection is also complex. Chen et al.'s MIP approach considers compatibility but requires complex modeling and lengthy solving. Santos et al.'s QCR hashing saves storage and enables efficient retrieval, but mainly targets numerical data and performs less effectively on text.

1.2.2.2 Fast Retrieval of Related Tables

To meet real-time query requirements over large numbers of data lake tables, researchers seek faster retrieval. Mainstream acceleration methods fall into index optimization and graph optimization. Common index techniques include locality-sensitive hashing and inverted-index search, which use approximation to accelerate discovery. Graph optimization constructs table-similarity graphs and performs approximate search on these graphs.

Hash indexes reduce the search space and substantially decrease the number of tables to inspect. D3L [2] builds hash indexes in a high-dimensional space to exclude tables with low similarity to the query, searching only tables whose hash values lie nearby. Nargesian et al. [3] use locality-sensitive hashing (LSH) to accelerate discovery based on set, semantic, and natural-language domains. Specifically, they bucket attributes using LSH to estimate joint distributions and efficiently compute unionability scores.

Estimating the intersection between a candidate set and a query set before deciding how to search also accelerates discovery. Overlap set similarity search measures similarity by intersection size. Unlike other measures, it is not biased against small sets and has therefore been applied to related table discovery in data lakes. Zhu et al. [5] propose JOSIE (Joining Search using Intersection Estimation), an algorithm for large sets and dictionaries. It estimates intersection size and dynamically decides whether to read the candidate set directly or read inverted-index lists, minimizing data-reading costs and improving search efficiency.

HWNS (Hierarchical Weighted Network Summarization) is a multilayer graph structure. Connections between vertices depend on the similarity of the tables they represent. The first layer is the simplest, with the fewest vertices and edges and the largest distances between vertices. The bottom layer is the densest, with the most vertices and edges and the smallest distances. Layer-by-layer traversal progressively approaches the nearest vertex to a query point. Fan et al. [14] apply the HNSW index to accelerate table retrieval. They first evaluate column similarity through cosine similarity between column embeddings and then determine table similarity using weighted bipartite matching.

These approaches still have shortcomings. D3L's high-dimensional hash indexes can suffer from hash collisions, high maintenance costs, and sensitivity to distribution changes. LSH involves difficult parameter selection and has limited ability to handle high-dimensional data. JOSIE may produce intersection-estimation errors, has limited handling of small sets, and incurs high inverted-index maintenance costs. Although HWNS accelerates discovery, constructing it is complex and consumes substantial memory.

1.2.3 Table Alignment and Missing-Value Repair

After top-k tables have been retrieved, joining or unioning them with a user's query table may introduce column-name and schema conflicts, producing null values that substantially reduce augmentation quality. This section reviews table alignment and missing-value repair methods intended to improve that quality.

Research on automated table joins focuses on reducing missing values and increasing automation. Khatiwada et al. [13] develop DIALITE, which uses ALITE to resolve conflicts between an original table and top-k candidates. Based on full disjunction, it first performs global schema matching to identify matching columns and label them consistently, then executes full disjunction to integrate the tables. This approach preserves accuracy, improves automation, and reduces missing values.

Chepurko et al. [10] develop ARDA for end-to-end automated table joins. A soft-key mechanism handles missing keys, while coreset techniques improve efficiency. However, ARDA relies on external systems such as Aurum to identify joinable tables, limiting its automation. Auctus [12], by comparison, integrates a dataset search engine and automatically performs joins and table expansion, giving it distinctive data-processing advantages.

Dong et al.'s PEXESO [8] expands base-table information through left joins. It first samples rows from the query table and then performs left joins on joinable tables. A major drawback is that left joins may produce many missing values. Zhu et al. [17] attempt to reduce missing values by transforming table data into numerical types and identifying statistically correlated columns. Nonetheless, mismatched columns may remain, and the resulting null values can impair subsequent processing.

Table-joining techniques have achieved automated processing, but existing methods generally support only equijoins and particular data types. When processing semantic data, their lack of semantic understanding leads to many null values in the augmented table, directly affecting downstream machine learning training. Effective imputation is therefore essential. Stekhoven et al. [15] propose random-forest-based imputation, iteratively predicting and filling missing values. Unlike simple random imputation, this model considers existing table features and produces reasonable replacements without disrupting the table's original characteristics.

Yoon et al. [16] propose GAIN, a missing-data imputation method based on generative adversarial networks. Its generator (G) fills missing data, while its discriminator (D) distinguishes real data from imputed data. GAIN also introduces a hint vector that provides partial information about missingness, helping the discriminator assess imputed values more accurately and addressing the generator's potential inability to learn the true data distribution.

Existing alignment and repair methods still have limitations. DIALITE integrates intercolumn relationships effectively, but full-disjunction processing causes scalability problems for large datasets. ARDA improves joining flexibility through soft keys, but external dependencies reduce automation. Auctus provides end-to-end automated joins, but has limited compatibility with different table types. PEXESO accelerates joining through row sampling, yet cannot fully eliminate null values.

Imputation methods also have their own limitations. Nonparametric random-forest methods can discover latent patterns in tabular data, but model training is complex and strongly dependent on features.

GAIN uses adversarial networks and hint vectors to strengthen the discriminator, but faces unstable GAN training and dependence on high-quality labeled data.

1.3 Work Presented in This Thesis

To provide an efficient and convenient table augmentation solution, this thesis designs and implements a fully automated system for data lake tables. The system has a web frontend for information presentation and interaction, and a backend for processing input and performing augmentation. Four core components - table preprocessing, related table discovery, table augmentation, and augmentation evaluation - form the automated workflow.

This thesis explains the overall architecture, implementation details, and core algorithms. Evaluation compares the system with established retrieval methods, including BM25, TF-IDF, and D3L, using multiple quantitative metrics. The experimental results confirm the proposed method's advantages in related table discovery.

Tests with actual tables show substantial performance advantages. In random table-vector generation scenarios, the HNSW-based method achieves over 90% accuracy using only 35% of the retrieval time required by brute-force traversal. On the Santos dataset, Embedding-HNSW achieves over 85% accuracy. In a test environment containing 3,000 tables, a single augmentation operation completes within 60 seconds, meeting practical requirements.

The main contributions are as follows:

(1) A complete, fully automated table augmentation system for data lakes is constructed and implemented. It automatically vectorizes tables, discovers related tables, executes augmentation, and imputes missing values, connecting the entire workflow.

(2) Systematic experiments compare the proposed Embedding-HNSW discovery approach with BM25, TF-IDF, D3L, and other established retrieval methods. Quantitative evaluation across multiple dimensions shows that the proposed method outperforms the baselines in related table discovery.

(3) Ablation experiments show that the Transformer trained through contrastive learning better captures relationships between table columns and improves discovery accuracy. Augmentation of open-source tabular datasets is also used to evaluate the system's augmentation effectiveness.

1.4 Organization of the Thesis

The chapters are organized as follows.

Chapter 1 introduces the current state of research on data lake table augmentation and describes the significance and scope of this project.

Chapter 2 introduces the technologies used in the system, focusing on the pretrained language models applied in its various components and their advantages.

Chapter 3 presents the detailed system design and implementation, including the architecture, processing workflow, and key algorithms.

Chapter 4 presents functional and performance testing, together with comparative analysis of the results.

Chapter 5 summarizes the work and proposes directions for future improvement.

Chapter 2 Related Technologies

This thesis designs and implements a semantically driven table augmentation system that enables intelligent augmentation of data lake tables. Its workflow contains several key stages, each facing distinct challenges in ensuring augmentation quality and a good user experience.

During preprocessing, heterogeneous data lake tables must be converted into consistently structured embedding vectors to facilitate efficient similarity computation. During discovery, the system must accurately capture semantic relationships between tables. During imputation, it must analyze table features in depth to fill missing values correctly.

The system addresses these challenges using three key technologies: an embedding model for text representation, a Transformer for column-semantic modeling, and BERT for missing-value imputation. This chapter introduces these models and analyzes their roles and advantages.

2.1 Embedding Model

2.1.1 Overview

An embedding model is a natural language processing model that converts text into high-dimensional numerical vectors. It generates these vectors by analyzing contextual semantic relationships, placing texts with similar meanings close together in the vector space and transforming discrete text into continuous mathematical representations. Embedding models support a wide range of tasks, including semantic search and text classification. Compared with traditional methods, they handle polysemy effectively and capture semantic relationships between words accurately [18][21].

2.1.2 Advantages

For similar-text search, embedding models offer the following advantages over direct text processing:

(1) Better generalization: processing text vectors addresses polysemy and synonymy. Measuring semantic relevance through vector similarity rather than text matching better captures table similarity.

(2) Lower storage requirements: embedding models compress sparse, discrete text into dense, high-dimensional vectors while retaining core semantic features.

(3) More efficient retrieval: vector similarity computation has much lower time complexity than direct text matching. It suits massive datasets and supports distributed computation and index optimization.

Traditional matching methods struggle with polysemy and synonyms in related table discovery. Embedding models replace textual matching with vector similarity computation and identify semantic relationships more accurately. They also reduce storage use and accelerate computation.

These advantages make fast retrieval over large numbers of data lake tables possible. For example, HNSW [14] uses table vectors to compute similarity and improves large-scale nearest-neighbor retrieval through appropriate index design and optimized neighbor computation, allowing the system to efficiently identify the tables most relevant to a query.

2.2 Transformer Models and Contrastive Learning

2.2.1 Overview

A Transformer is a neural network architecture based on self-attention, using stacked encoder and decoder modules. Its four core components are multihead self-attention, feedforward networks, residual connections, and positional encoding [19].

Contrastive learning is a self-supervised method that learns effective feature representations by comparing sample similarity. Its central idea is to bring similar samples closer and push dissimilar samples apart. Common methods include SimCLR [20]. Contrastive learning is widely applied in images, text, and multimodal tasks and is an effective form of unsupervised learning.

2.2.2 Advantages

Transformers use multihead attention to capture semantic relationships among input texts. For table semantics, this mechanism automatically identifies and captures relationships between different columns. Because a Transformer models contextual information independently, intercolumn relationships need not be manually predefined. Self-attention therefore suits unsupervised tasks such as table similarity computation.

Contrastive learning is an efficient unsupervised method. Data augmentation constructs positive and negative sample pairs, avoiding extensive manual annotation. Training a Transformer through contrastive learning also improves its ability to capture fine-grained semantic differences. For tabular data, this further optimizes column-semantic feature extraction, strengthens semantic learning, and significantly improves discovery accuracy.

Thus, Transformers provide strong automated modeling of column semantics, while contrastive learning improves the discrimination of different meanings. Their combination supports the exploration of deep semantic relationships and related table discovery.

2.3 BERT Model

2.3.1 Overview

BERT (Bidirectional Encoder Representations from Transformers) is a pretrained language model proposed by Google.

Based on the Transformer architecture, BERT uses a bidirectional encoder to model context. It is commonly applied to natural language processing tasks such as question answering, natural language inference, and text classification [18].

2.3.2 Advantages

Compared with other models for imputing missing text, BERT offers the following advantages for missing values in tables:

(1) Strong contextual understanding: BERT is trained through masked language modeling (MLM), enabling it to use context on both sides when predicting missing words. Unlike conventional left-to-right or right-to-left unidirectional language models, it captures contextual semantics more comprehensively and handles missing values more accurately [18].

(2) Suitable architecture: MLM randomly masks some input tokens and predicts their original content, closely matching table imputation. Missing values can naturally be treated as masked inputs, allowing BERT to be applied directly without architectural changes [18].

BERT's contextual modeling and masking mechanism therefore offer clear advantages. Its bidirectional encoder uses information before and after a missing value to improve inference accuracy. Its training objective naturally matches imputation, allowing direct use without special modifications. Compared with traditional methods, BERT provides more accurate and semantically consistent replacements, making it an efficient solution for completing missing table values.

2.4 Chapter Summary

This chapter introduces the core technologies of the semantically driven table augmentation system. The embedding model plays a key role in similarity computation, improving generalization and efficiency and supporting large-scale retrieval. The combination of Transformers and contrastive learning improves intercolumn semantic modeling and enables finer-grained similarity computation. Finally, BERT's contextual modeling and masking mechanism improve imputation accuracy. Together, these technologies form the system's modeling foundation.

Chapter 3 Detailed System Design and Implementation

The semantically driven table augmentation system aims to efficiently discover data lake tables and intelligently augment a user's query table. Its automated workflow covers table input, semantic modeling, related table discovery, augmentation-suggestion generation, and missing-value imputation. These functions improve augmentation quality and the quality of the resulting table, supporting downstream analysis.

The system contains five key processing steps: preprocessing, storage management, similar-table retrieval, augmentation execution, and imputation. This chapter first analyzes functional requirements for preprocessing, storage, discovery, execution, and imputation. Based on the preceding technological framework, it then describes the architecture, operating environment, and overall workflow. Finally, it details the design and implementation of the preprocessing, discovery, execution, and imputation modules, including their internal logic and algorithms.

3.1 Requirements Analysis

3.1.1 Table Preprocessing

Functional requirements: preprocessing provides efficient representation and similarity modeling for data lake tables. The system first uses a pretrained embedding model to vectorize all tables, converting each into a representation of a specified length. It then obtains table similarity from vector similarity and constructs an HNSW neighbor graph for efficient retrieval and discovery.

Performance requirements: the module should be stable and usable, maintaining efficient performance for different table sizes. Vectorization must accurately extract semantic features within a reasonable time and encode each column as a fixed-length vector. Similarity computation and HNSW construction should have acceptable time and space complexity.

3.1.2 Related Table Discovery

Functional requirements: this module efficiently identifies data lake candidates highly similar to the user's query table. Given the table and query parameters, including the return count k, it retrieves top-k tables from the constructed HNSW graph, returns them in similarity order, and supplies them to downstream augmentation.

Performance requirements: because a data lake may contain many tables, the module must minimize query time while retaining high retrieval accuracy to ensure responsiveness and a good user experience.

HNSW construction and search involve approximation and some randomness, so strict consistency and persistence of query results are not required. Nevertheless, the module must remain stable and offer good query performance, balancing practical efficiency and accuracy.

3.1.3 Augmentation Execution

Functional requirements: after related tables are identified, the execution module augments the user's query table using these tables. It should apply the extracted parameters and LLM-generated suggestions accurately and reasonably and output the augmented table.

Performance requirements: this module must be highly reliable. Because LLM output is uncertain, it must strictly process output according to predefined format specifications and use suggestions appropriately to ensure compliant results. Comprehensive exception handling must address problems during augmentation and ensure correct output and stable operation.

3.1.4 Missing-Value Imputation

Functional requirements: the augmented table may contain missing values. The module must identify column data types correctly and select appropriate imputation methods. It should preserve the original table's semantic and structural features as far as possible, avoiding semantic changes that impair downstream machine learning training.

Performance requirements: imputation must be efficient and reliable. The module must accurately identify column types and apply suitable strategies. Because missing text requires calls to a pretrained BERT model, these calls must be stable and their results accurate. Imputation speed must remain acceptable without sacrificing quality.

3.1.5 Data Storage

Functional requirements: this module provides unified storage and maintenance of original data lake tables, vectorized embeddings, and all user-related data. A unified interface should support insertion, updating, deletion, and querying, encapsulating these operations so other modules can access and manipulate data efficiently and reliably.

Performance requirements: data management must be highly reliable, with strict guarantees for integrity, consistency, persistence, and fault tolerance. During system faults and other abnormal conditions, data must not be lost or damaged, and stable services must remain available.

3.2 Architecture Design

The system uses a four-layer architecture: the interaction interface layer, business-processing layer, data-storage layer, and physical-resource layer. Figure 3-1 shows the prototype architecture. The interface layer manages user interaction through login/registration, table augmentation, and downstream evaluation pages, guiding users through augmentation and assessment. The business layer is the system's core and contains related table discovery and augmentation modules. For discovery, pretrained Jina Embedding and column-contextual Transformer models convert uploaded tables into embeddings; the prebuilt HNSW graph then retrieves top-k tables online. For augmentation, the system calls an LLM remotely to augment tables and impute missing values based on relevant information. The storage layer holds data lake tables and user data. Vector representations stored during preprocessing substantially shorten online query times and improve user experience. The physical layer includes processors, disks, and other hardware supporting operation.

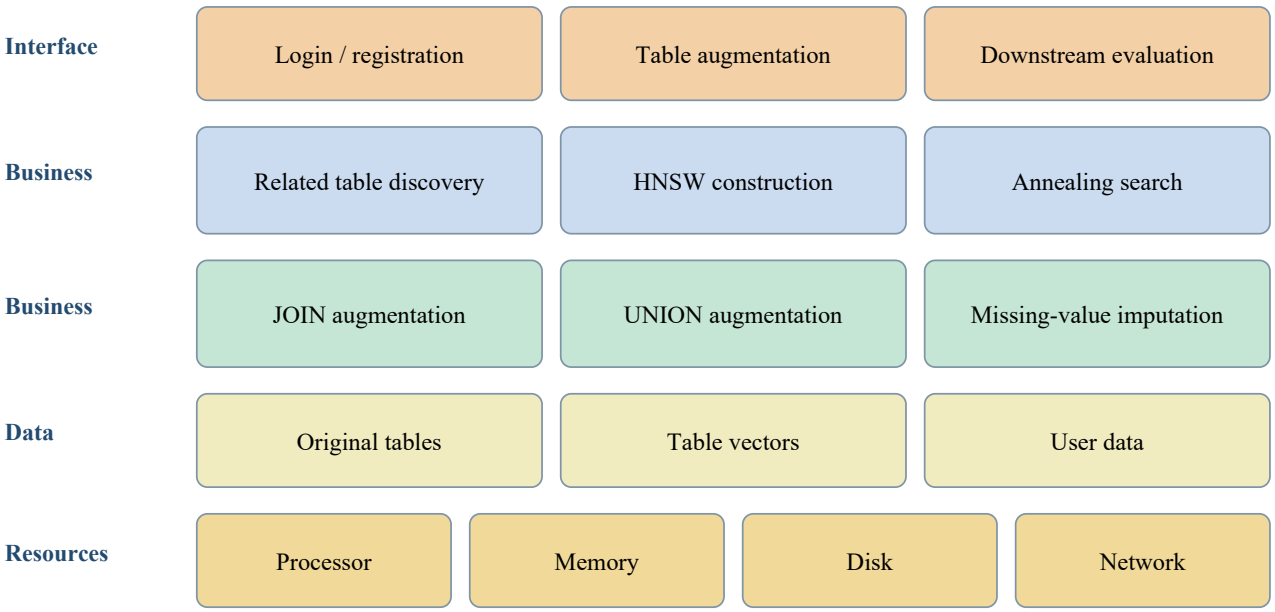


Figure 3-1. System architecture design.

Deployment separates the frontend and backend. The Vue3 frontend presents pages and handles interaction; the FastAPI backend implements business logic, stores data, and provides interfaces. All required pretrained models are deployed in the local backend, and mainstream libraries such as PyTorch and Pandas are used for implementation.

Backend automation follows four table-processing stages: preprocessing, discovery, augmentation execution, and missing-value imputation.

The backend architecture in Figure 3-2 is divided into preprocessing and ad hoc user-query processing.

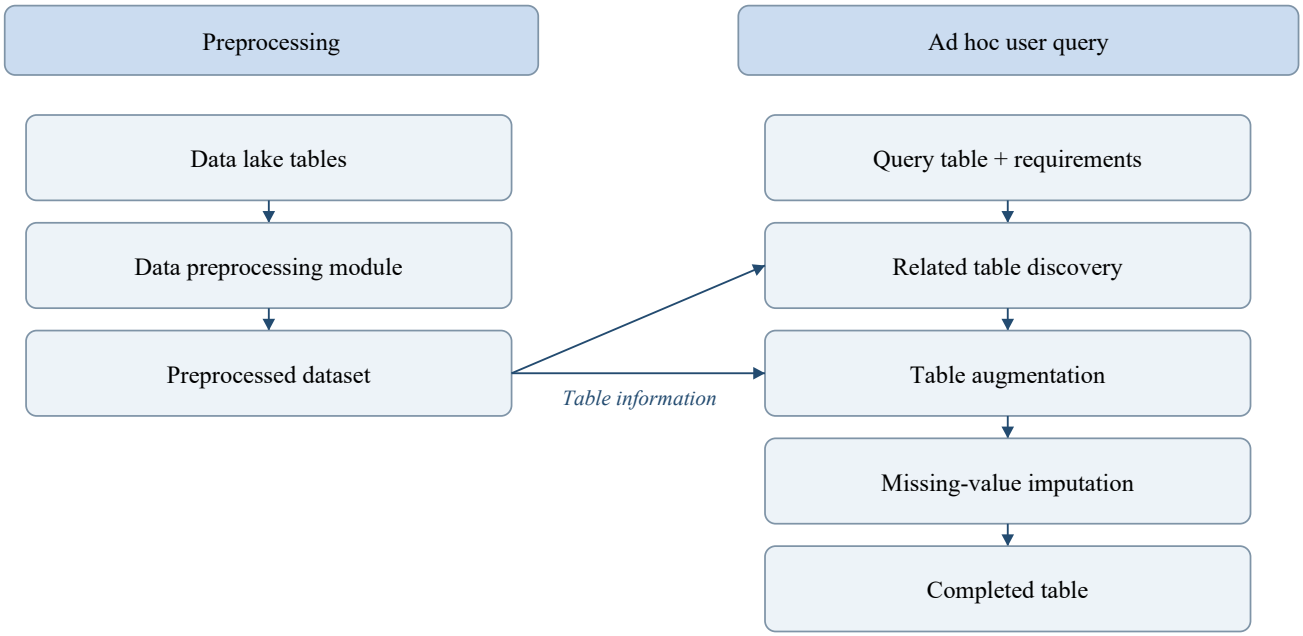


Figure 3-2. Backend system architecture.

The backend consists mainly of three functional modules: data preprocessing, related table discovery, and table augmentation. To automate the entire workflow, initialization automatically vectorizes all data lake tables and constructs an HNSW index graph.

During ad hoc querying, a predefined workflow coordinates these modules. The discovery module first searches the prebuilt HNSW graph for tables related to the query and passes their information to the augmentation module. The latter combines user requirements, related-table information, and LLM suggestions to augment the query table, then fills missing entries. The completed table is returned as the final result.

3.3 Workflow Design

3.3.1 Overall Workflow

Figure 3-3 presents the backend software workflow and its implementation components. These mainly include MySQL for the database, an embedding model for vectorization, Pandas and an LLM for augmentation, and BERT for imputation.

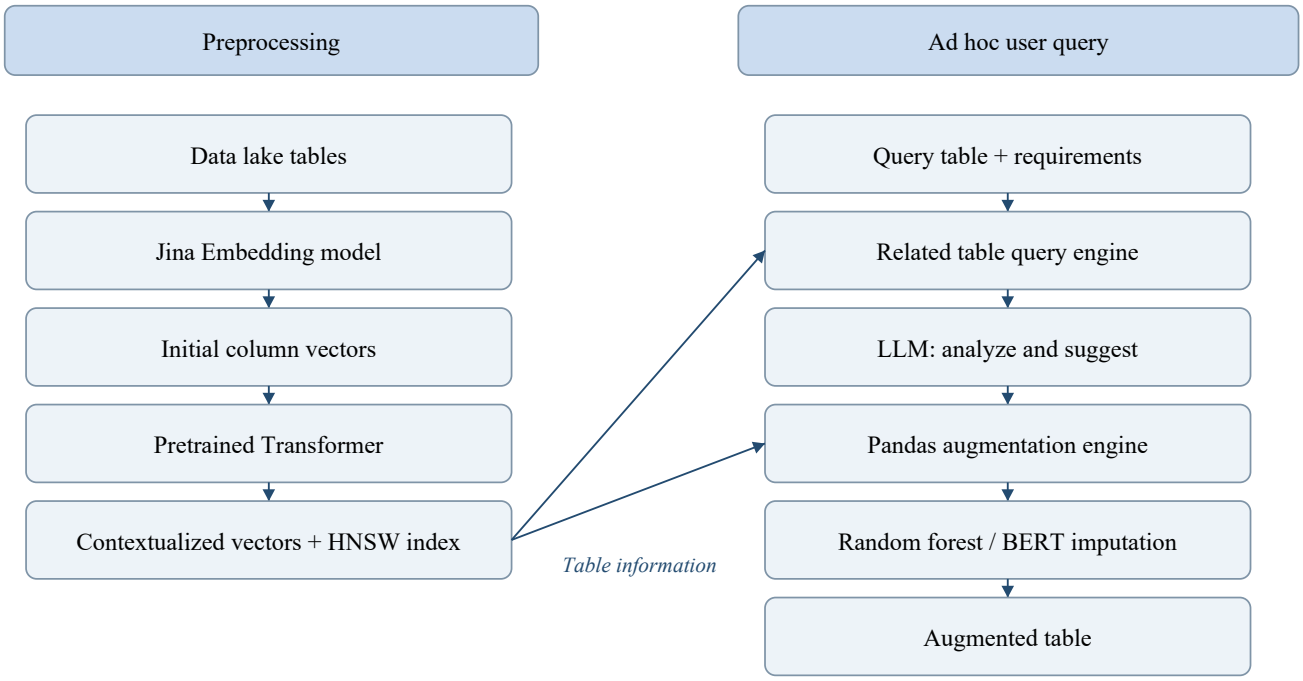


Figure 3-3. Backend software workflow.

3.3.2 Table Preprocessing

3.3.2.1 Overview

The preprocessing module runs at the beginning of system operation and converts table data into vector arrays in two transformations. Text vectorization requires an embedding model, while modeling relationships between columns requires a Transformer. These two models therefore form the module's core.

Jina Embeddings, proposed by Jina AI, converts text into high-dimensional vectors. Based on the Transformer architecture, it combines contrastive and multitask learning during training and performs well on semantic text matching and vector retrieval. It accepts up to 8,192 input tokens and offers good inference speed and deployability, making it suitable for production applications [21].

Semantic relationships between tables are often implicit and depend on relationships between columns within a table, making explicit modeling difficult. Conventional supervised learning is therefore difficult to apply directly when training the intercolumn Transformer. The system instead constructs positive and negative pairs for contrastive learning, enabling automatic learning of relationships between column vectors without explicit labels.

Training uses normalized temperature-scaled cross-entropy (NT-Xent) loss. This strengthens discrimination between positive and negative pairs. Combined with the Transformer's multihead self-attention [19], it captures deeper semantic relationships between column vectors.

This improves the accuracy of table-semantic representations and, consequently, related table discovery.

Once vector arrays have been obtained, a custom table-distance formula computes overall similarity between tables. The resulting similarities are used to construct an HNSW graph. When a query table is submitted, the system searches this graph using a strategy resembling simulated annealing, producing high-quality related tables for subsequent augmentation.

3.3.2.2 Table Column Encoding

The initial encoding method concatenated every row's content in a column, separated by spaces, and supplied the resulting text to the embedding model. This has an obvious drawback: typical pretrained embedding models have input-length limits, often 512 tokens. Although Jina Embedding v2 base-en supports 8,192 tokens [21], some columns still exceed that limit and may contain tens of thousands of tokens.

To preserve column semantics while respecting the input limit, the system uses TF-IDF-based content truncation. It first traverses all tables and computes global inverse document frequency (IDF). For every row of a column, the sum of its TF-IDF scores is used as its row score. Rows are sorted by decreasing score and concatenated until the model's maximum token length is reached; remaining content is ignored.

This controls input length while maintaining embedding quality and preserving important content, providing a foundation for column-vector representation and similarity modeling. Algorithm 1 details the process.

Algorithm 1. Obtain the Text Representation of a Table

Input: Table T; word-score dictionary IDF; maximum token input length m
Output: Column texts column_texts

```
for each row r in T:
    row_score[r] = sum(IDF[token] for token in r)
column_texts = []
max_length = max(len(column_texts[c]) for column c in T)
while max_length < m:
    r = row with the highest row_score
    for each cell c in r:
        column_texts[name(c)].append(c)
    row_score[r] = 0
    update max_length
return column_texts
```

To verify the effectiveness of TF-IDF in preserving semantic features, experiments compare TF-IDF-selected rows with randomly selected rows on three datasets. The accuracy of the resulting embeddings in related table discovery is shown in Table 3-1.

Table 3-1. Accuracy comparison of table truncation algorithms.

Dataset	Candidate count	TF-IDF	RANDOM
Santos small	1	1.000	0.980
	5	0.812	0.801
	10	0.732	0.724
WWk small	1	0.035	0.036
	5	0.295	0.245
	10	0.427	0.389
WWk large	1	0.036	0.035
	5	0.245	0.200
	10	0.386	0.369

Table 3-1 shows that TF-IDF generally performs better than random selection across the three datasets, indicating that it more effectively preserves semantics while reducing input length.

Finally, each column's selected text is supplied to the embedding model, producing a vector array for every table.

3.3.2.3 Contrastive Learning

To capture intercolumn semantic features more effectively while accounting for their implicit and relational nature, this thesis combines a Transformer with contrastive learning. Training aims to reduce the distance between semantically identical or similar tables and increase the distance between dissimilar tables.

An augmentation operation E(s, n) is defined for an individual table. For every data lake table t in T, an augmented copy is generated, yielding an original table t_ori and an augmented table t_aug. They are converted into vector representations Z_ori and Z_aug using the preceding method. Algorithm 2 describes augmentation, and Figure 3-4 compares a table before and after augmentation.

Algorithm 2. Augment a Single Table

Input: Table T; number of cell swaps s; noise level n
Output: Augmented table T

```
for i from 1 to n:
    c = randomly selected column from T
    c1, c2 = two randomly selected cells from c
    swap c1 and c2
for each column c in T:
    if type(c) is numeric:
        inject noise into c according to noise level n
return T
```

country	capital	population
China	Beijing	1,412,000,000
Canada	Ottawa	40,000,000
Japan	Tokyo	125,000,000



country	capital	population
China	Beijing	1,412,000,123
Japan	Ottawa	40,000,456
Canada	Tokyo	125,000,789

Figure 3-4. Comparison before and after table augmentation.

The contrastive augmentation process then trains the Transformer using a contrastive loss, as shown in Algorithm 3.

Algorithm 3. Train the Column-Contextual Transformer

Input: Table vectors Z; augmented vectors Z'; learning rate theta; batch size b; epochs e
Output: Trained model

```
model = Transformer model with random parameters
for epoch from 1 to e:
    randomly split Z, Z' into corresponding batches B, B'
    for each corresponding batch pair B, B':
        loss = L_contrast(Z_B, Z'_B')
        model = backward(model, theta, gradient(loss, model))
return model
```

The loss is defined as follows. It uses cross-entropy for an original batch B and an augmented batch B'. For vectors z_i and z'_i originating from the same table, the pairwise loss l(z_i, z'_i) is defined by Equation (3.1).

$$l(z_i, z'_i) = -\log [\exp(simi(z_i, z'_i) / t) / D_i] \quad (3.1)$$
$$D_i = \Sigma_{k=1, k\neq i}^{b} \exp(simi(z_i, z_k) / t) + \Sigma_{k=1}^{b} \exp(simi(z_i, z'_k) / t)$$

In Equation (3.1), simi is the table-similarity function and t is a temperature hyperparameter. The original text states that a smaller loss is obtained by minimizing the similarity score between a table's original and augmented vectors and maximizing the score between vectors of different tables.

The preceding sentence retains the wording of the source; its stated optimization direction is inconsistent with the contrastive-learning objective described earlier.

The total batch loss L_contrast is given in Equation (3.2).

$$L_{contrast} = (1 / 2b) \Sigma_{k=1}^{b} [l(z_k, z'_k) + l(z'_k, z_k)] \quad (3.2)$$

3.3.2.4 Transformer Model for Column-Semantic Relationships

The proposed Transformer converts column-independent vector arrays into column-contextual vector arrays. A six-hidden-layer Transformer is specifically designed and trained through contrastive learning to model intercolumn semantics. Figure 3-5 shows the architecture.

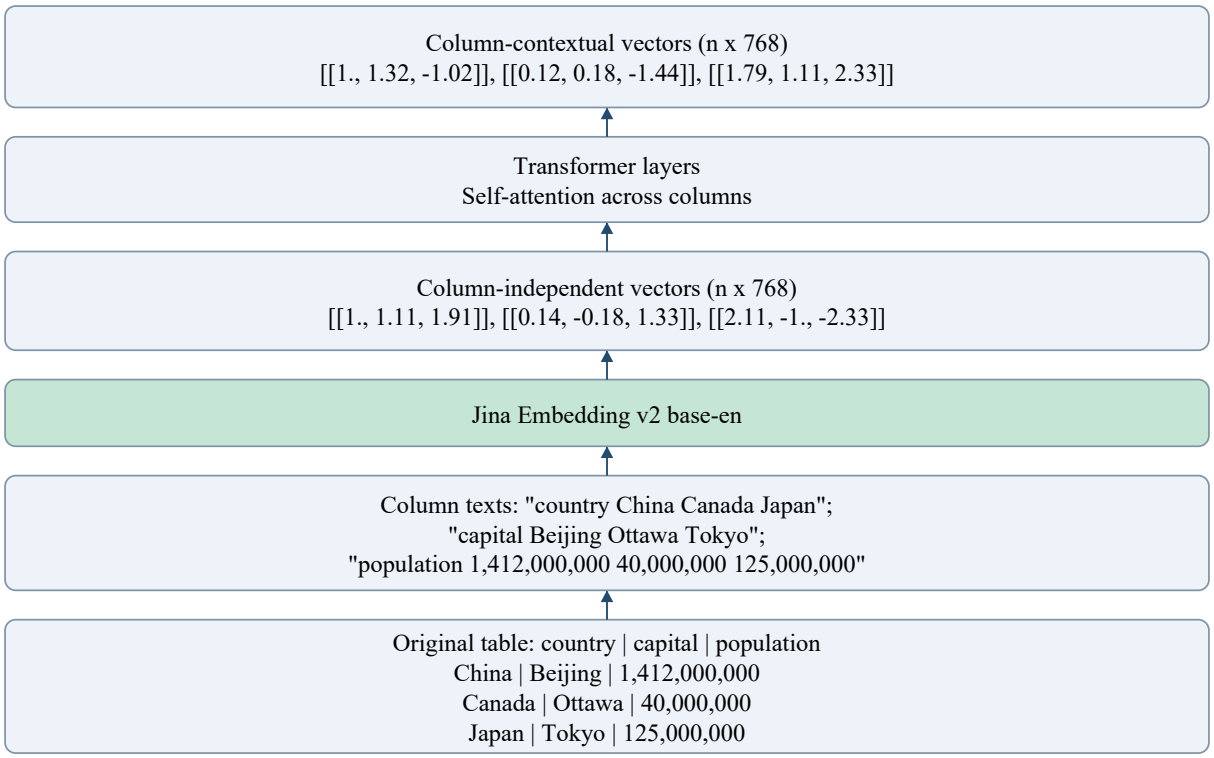


Figure 3-5. Model architecture.

The input is a two-dimensional array of shape n x 768, where n is the number of columns. After passing through the encoder and decoder, the output has the same shape.

The Transformer's self-attention captures relationships between columns within a table and generates column-contextual vectors, improving discovery accuracy.

3.3.2.5 Table Similarity

Similarity computation is central to identifying related tables in both preprocessing and discovery. To account for the semantics of all columns and obtain an overall score, this thesis models similarity as weighted bipartite matching combined with a greedy algorithm. For any two tables S and T, a bipartite graph G = (S, T, E) is constructed, where S and T represent their columns and E represents similarity scores between column pairs. To prevent unrelated columns from affecting the overall score, the edge weight E(s, t) is defined by Equation (3.3).

$$E(s, t) = \text{cosine_similar}(T_{i,a}, T_{j,b}), \text{ if } \text{cosine_similar}(T_{i,a}, T_{j,b}) \geq \beta;$$
$$\text{otherwise, the edge does not exist.} \quad (3.3)$$

After constructing the graph, similarity is calculated using Equation (3.4). Dividing by the larger column count removes the advantage of tables with more columns.

$$\text{Simi}(S, T) = \max_{\text{match}} G(S, T) / \max(|S|, |T|) \quad (3.4)$$

Figure 3-6 illustrates this calculation. The filtering threshold is 0.4. Since the similarity between columns S3 and T3 is below 0.4, that edge is discarded. Selected matches are highlighted in red, yielding a table-similarity score of (0.8 + 0.7 + 0.65) / 4 = 0.5375.

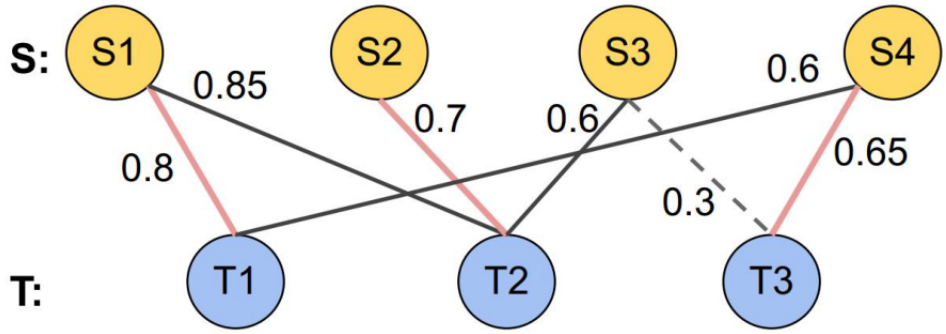


Figure 3-6. Example of table-similarity computation.

Direct weighted bipartite matching has high time complexity: O(n^4) when converted to a network-flow problem using Dinic's algorithm. To simplify computation, this thesis uses Sinkhorn-Knopp.

This reduces the time complexity of computing one table-pair similarity to O(n^3).

Sinkhorn-Knopp is an iterative method that alternately normalizes the rows and columns of a nonnegative matrix to obtain a doubly stochastic matrix. Through repeated normalization, it approaches a solution under doubly stochastic constraints. Each row and column of such a matrix sums to 1, making it useful for maximum bipartite matching. Transforming the original similarity matrix balances row and column weights and enables approximate computation of a maximum-matching score [22]. Algorithm 4 presents the calculation.

Algorithm 4. Compute the Similarity Score of Two Tables

Input: Tables S, T; threshold beta; Sinkhorn-Knopp iterations num_iter
Output: Table-similarity score

```
for i from 1 to |S|:
    for j from 1 to |T|:
        similarity_matrix[i][j] = cosine_similarity(S[i], T[j])
set entries below beta to 0
for k from 1 to num_iter:
    for i from 1 to |S|:
        for j from 1 to |T|:
            similarity_matrix[i][j] /= sum(similarity_matrix[i][*])
            similarity_matrix[i][j] /= sum(similarity_matrix[*][j])
score = 0
for k from 1 to min(|S|, |T|):
    i, j = position of maximum entry in similarity_matrix
    score += similarity_matrix[i][j]
    similarity_matrix[i][*] = 0
    similarity_matrix[*][j] = 0
score /= max(|S|, |T|)
return score
```

3.3.2.6 HNSW Graph Construction

HNSW (Hierarchical Navigable Small World) is a graph-based algorithm for approximate nearest-neighbor search. It uses multiple sparse graph layers for efficient search: upper layers provide coarse navigation, while lower layers provide finer search, progressively approaching the optimal result. The graph reduces traversal complexity while maintaining high efficiency and accuracy.

Traditional HNSW primarily searches for nearest neighbors between vectors. Here, table similarity uses a custom function, so the multilayer structure is modified accordingly. The NSW structure is based on minimum spanning trees. First, data points are probabilistically assigned to layers. Second, a minimum spanning tree is built within each layer, and short and long edges are added to strengthen connectivity. Finally, nodes with the same identifiers are connected across layers.

These connections form navigation paths between layers. The design retains fast, accurate HNSW search while accommodating custom similarity computation. Algorithm 5 gives the implementation.

Algorithm 5. Construct the HNSW-MST Search Graph

Input: Similarity matrix; table count n; graph layer count m; hyperparameter k
Output: HNSW-MST graph

```
graph = []; layer = []
for i from 1 to n:
    for j from 1 to m:
        if rand(0, 1) <= k^(m-j):
            layer[i] = j
            break
for j from 1 to m:
    current_points = {i in [1, n] where layer[i] == j}
    g = minimum spanning tree of current_points
    for each e in current_points:
        if e is not in g and e is a short edge:
            add e to g
        else if e is not in g and e is a long edge:
            add e to g
    add g to graph
return graph
```

3.3.3 Related Table Discovery

3.3.3.1 Overview

This module searches the HNSW graph using table similarity. Since traversing all tables is inefficient, a multilayer nearest-neighbor strategy is used. First, layer-by-layer search descends to the bottom layer and quickly obtains scores for most likely candidates, avoiding a costly linear scan. Second, all candidates are ranked by similarity, and the highest-scoring k tables are returned. Together, these steps maintain accuracy while substantially improving efficiency.

3.3.3.2 Algorithm Workflow

The system randomly selects a starting node in layer 1 and calculates its similarity distance to the query. Within each layer, a strategy resembling simulated annealing moves to the neighboring node with the smallest distance, approaching a local optimum. After searching the current layer, the closest node found is used as the starting point for the next layer. This continues down to the bottom layer. Finally, the k nodes closest to the query are returned as related tables. Algorithm 6 presents the pseudocode.

Algorithm 6. Discover Related Tables

Input: Query q_table; data lake tables; HNSW graph g; return count k
Output: Related-table indices

```
distance = [INF]
start = randomly selected node from layer 1
distance[start] = Simi(q_table, tables[start])
for i from 1 to g.m:
    steps = |g.layer[i]| / 5
    while steps > 0:
        for each v in neighbour[start]:
            distance[v] = Simi(q_table, tables[start])
        start = v in neighbour[start] with minimum distance[v]
        steps -= 1
return indices of the top-k minimum entries in distance
```

3.3.4 Table Augmentation

3.3.4.1 Overview

The augmentation module has two submodules: augmentation execution and missing-value imputation. During operation, it uses related tables and user requirements to appropriately augment the query table.

For execution, the query table, related tables, and extracted parameters are supplied to an LLM to obtain suggestions. Pandas operations then follow a JOIN-before-UNION sequence to produce an augmented table. Format validation and exception handling address uncertainty in LLM output and ensure stable processing and correct results.

For imputation, the numeric strategy is determined from augmentation parameters. Each column's type is then inspected: numerical fields use the selected strategy, while text fields use pretrained BERT and contextual semantics. This improves completeness while preserving the original structure and meaning as much as possible.

3.3.4.2 Augmentation Execution

Unlike traditional methods, this system introduces an LLM for more intelligent augmentation decisions. It first extracts key information from the query and related tables, including headers and sample data, and supplies it with parameters to the LLM. The LLM then outputs a strategy in a predefined format containing JOIN and UNION suggestions. The system retrieves and parses those suggestions and applies predefined augmentation logic to expand the table semantically and structurally.

The predefined suggestion format is a JSON object with two fields: join_operations and union_operations. join_operations is a list of sublists, each representing a JOIN. A sublist has the form ["query column", "related-table ID.related column", "retained column 1", "retained column 2", ...]. The first element is the query-table join column, the second is the related-table join column including its table ID, and the remaining elements are related-table columns to retain in the final table.

union_operations is a list with an even number of elements. Every pair of adjacent sublists represents one UNION. The first specifies source columns in the form "related-table ID.column name"; a plus sign (+) may combine multiple columns. The second specifies the corresponding column names in the final table in the same order.

An example suggestion is:

```
{
  "join_operations": [
    ["department", "1.department", "department", "established date"]
  ],
  "union_operations": [
    ["2.staff name", "2.department"],
    ["name", "department"]
  ]
}
```

In join_operations, ["department", "1.department", "department", "established date"] specifies a join between the query's department column and the department column in related table 1, retaining department and established date from that table. In union_operations, the first sublist ["2.staff name", "2.department"] selects staff name and department from related table 2. The second, ["name", "department"], names them name and department in the final table.

The system executes JOIN operations first, parsing the query column, related-table ID, join column, and retained columns for each suggestion. Pandas merge joins the tables and redundant columns are removed. UNION operations are then processed in pairs and new column names are assigned consistently. Data are extracted from related tables and concatenated; the concatenated result is combined with the original query table to complete augmentation. Algorithms 7 and 8 show the JOIN and UNION pseudocode.

Algorithm 7. Execute JOIN Augmentation

Input: query_table; join_operations; related_tables
Output: Augmented query_table

```
for each join_op in join_operations:
    query_column = join_op[0]
    related_ref = join_op[1]
    table_id, related_column = ParseTableRef(related_ref)
    keep_columns = Union(join_op[2:], [related_column])
    related_table = LoadCSV(related_tables[table_id])
    related_table = FilterColumns(related_table, keep_columns)
    related_table = DropDuplicates(related_table, related_column)
    query_table = MergeTables(query_table, related_table,
        on_left=query_column, on_right=related_column, method="outer")
    query_table = DropDuplicateColumns(query_table)
return query_table
```

Algorithm 8. Execute UNION Augmentation

Input: query_table; union_operations; related_tables
Output: Augmented query_table

```
for i from 1 to |union_operations|, step 2:
    operation_list = union_operations[i]
    column_names = union_operations[i + 1]
    involved_tables = ExtractTableIDs(operation_list)
    table_dfs = InitializeEmptyDictionary()
    for each table_id in involved_tables:
        table_dfs[table_id] = LoadCSV(related_tables[table_id])
    for j from 1 to |operation_list|:
        op = operation_list[j]
        new_col = column_names[j]
        if "+" is in op:
            ref1, ref2 = SplitJoinColumns(op)
            ApplyJoinBetween(table_dfs, ref1, ref2, new_col)
        else:
            tid, col = ParseTableRef(op)
            RenameColumn(table_dfs[tid], col, new_col)
    union_table = JoinUnionTables(table_dfs, operation_list)
    query_table = ConcatenateRows(query_table, union_table)
return query_table
```

3.3.4.3 Missing-Value Imputation

Field-alignment inconsistencies and missing data may leave empty cells after augmentation. To improve completeness and usability, the system implements an imputation module. It automatically identifies and fills missing fields according to the user's strategy, improving the augmented table's performance in downstream machine learning tasks.

Imputation has two main steps. First, the system attempts to convert each column to a numerical type for consistent subsequent processing. For nonnumeric columns, missing values are replaced with [MASK] and filled using pretrained BERT. Numerical columns follow the strategy in the augmentation parameters: prediction with a random forest regressor, or replacement with the mode or median. Algorithm 9 shows the pseudocode.

Algorithm 9. Impute Missing Values

Input: Table df to fill; strategy
Output: Filled table df

```
for each column in df:
    convert column to numeric type if possible
for each column in df:
    if column is not numeric:
        for each value in column where value is NA:
            filled_value = BERT_Predict(value)
            value = filled_value
for each column in df:
    if column is numeric:
        if strategy == "MODEL":
            use RandomForestRegressor to predict missing values
        else if strategy == "MEAN":
            fill missing values with the mode
        else:
            fill missing values with the median
return df
```

Text columns are filled before numerical columns because text often determines the meaning of numerical fields. Numerical values in structured tables frequently depend on text fields, so filling text first provides more complete context and improves numerical prediction accuracy.

3.3.5 Data Storage

3.3.5.1 Overview

The storage module manages two types of data: data lake table information and all user-related data. This section describes storage methods in relation to the preceding modules.

Storage uses two approaches: MySQL for structured data and the file system for unstructured or large files. MySQL is mainly used for many small records requiring responsive insertion, deletion, updating, and querying.

Examples include user information, data lake table metadata, and augmentation history. These data often require concurrent access and are efficiently managed in a relational database. The file system stores larger objects with less stringent operation-speed requirements, such as table vectors and user avatars, and better suits their capacity and access patterns.

3.3.5.2 Data Stored in MySQL

The database contains three tables: user information, data-table index information, and user augmentation-history information. Figure 3-7 shows examples.

#	username	password	avatar_path	user_type
1	[redacted]	[redacted]	[redacted]	user
2	admin	[redacted]	(NULL)	admin

(a) User information table.

#	table_id	table_path	pure_embedding_path	processed_embedding_path
1	1	E:/DLES_System/da...	E:/DLES_System/da...	E:/DLES_System/datalake_e..
2	2	E:/DLES_System/da...	E:/DLES_System/da...	E:/DLES_System/datalake_e..
3	3	E:/DLES_System/da...	E:/DLES_System/da...	E:/DLES_System/datalake_e..
4	4	E:/DLES_System/da...	E:/DLES_System/da...	E:/DLES_System/datalake_e..
5	5	E:/DLES_System/da...	E:/DLES_System/da...	E:/DLES_System/datalake_e..
6	6	E:/DLES_System/da...	E:/DLES_System/da...	E:/DLES_System/datalake_e..
7	7	E:/DLES_System/da...	E:/DLES_System/da...	E:/DLES_System/datalake_e..
8	8	E:/DLES_System/da...	E:/DLES_System/da...	E:/DLES_System/datalake_e..

(b) Data-table index information table.

#	username	history_tree
1	[redacted]	

(c) User augmentation-history information table.

Figure 3-7. Example database tables.

The user table stores account names, passwords, avatar URLs, and other basic information. It is used by login and registration and provides access control and authentication for other modules.

Data lake index information is used by preprocessing, discovery, and augmentation. Preprocessing writes an auto-incrementing table ID, the table's storage path, and its vector path to the table-information database. Discovery and augmentation query this information to quickly locate table or vector files.

The augmentation-history table contains user IDs and history information. Its history field is JSON, representing a hierarchical file-system structure.

When history is modified, this field is synchronized with the corresponding files.

3.3.5.3 Data Stored in the File System

The file system stores data lake tables, vectors, and user augmentation records. Data lake tables and their vectors are stored in folders with standardized names; user records are organized by username under "fixed-name folder/username".

Data lake tables must be prepared before operation. Vector generation has two steps: Jina Embedding produces initial vectors and temporarily stores them; the pretrained column-contextual Transformer then converts independent vectors into contextualized vectors and writes them to a folder for other modules.

During HNSW construction, the preprocessing module obtains vector paths from the database and loads files to compute table similarity and build the graph. Augmentation uses table IDs supplied by discovery to read the corresponding data lake tables through the index and execute operations.

User augmentation records are also stored in the file system. Each case has its own folder containing four files: the original table, its initial vectors, its contextualized vectors, and the conversation history between the user and LLM. Figure 3-8 shows an example.



Figure 3-8. Example of one table augmentation record.

3.3.6 Downstream Task Validation

3.3.6.1 Overview

A built-in validation module helps users compare performance before and after augmentation. It integrates a random forest model to assess uploaded data. Users upload tables and specify a target column; the system automatically splits the data into training and test sets, trains the model, and makes predictions. For numerical targets, root mean squared error (RMSE) measures the deviation between predictions and true values.

$$\mathrm{RMSE} = \sqrt{[(1 / n) \Sigma_{i=1}^{n} (y_i - \hat{y}_i)^2]}$$

For text targets, F1-score is calculated to quantify prediction performance.

$$\text{F1-score} = 2 \times \text{Precision} \times \text{Recall} / (\text{Precision} + \text{Recall})$$

Results are displayed as bar charts to help users understand the changes brought by augmentation. As this module is not the thesis's core focus, it is not discussed further here.

3.4 Chapter Summary

This chapter comprehensively presents the system's requirements, architecture, and backend design. It analyzes both functional and performance requirements for five key modules and specifies their needs.

Building on Chapter 2, it designs four main modules: preprocessing, discovery, augmentation, and imputation. HNSW graphs and LLM suggestions automate augmentation. It also identifies the technical framework and operating environment, including Python and Vue3, and explains the selection of components such as Jina Embedding and Qwen. Software flowcharts describe vectorization, semantic augmentation, and imputation.

The chapter then presents implementation logic for the three core modules. Preprocessing uses a pretrained embedding model and a fine-tuned Transformer to generate high-quality contextualized column-vector arrays, then constructs an HNSW graph using table similarity. Discovery uses a simulated-annealing search strategy across graph layers, repeatedly moving to the closest neighbor and approaching local optima to return top-k tables. Augmentation has two stages: executing LLM-generated suggestions in JOIN-before-UNION order, and selecting suitable numeric and text imputation methods according to user requirements and field types. These steps complete the augmentation workflow.

Chapter 4 System Testing and Analysis of Results

Based on the preceding design and implementation, this chapter comprehensively tests the functionality and performance of the Semantically-Driven Table Augmentation System for Data Lakes (SDASDL), verifying the effectiveness of the proposed methods and the feasibility of the theoretical results.

4.1 Test Objectives and Configuration

4.1.1 Test Objectives

The tests assess functional completeness, query efficiency, augmentation effectiveness, and overall processing speed in five areas.

(1) Functional testing: automated scripts simulate user operations across login/registration, related table discovery, augmentation, and downstream evaluation to verify correctness and consistency.

(2) Discovery performance: the proposed method is compared with BM25, TF-IDF, and other baselines on three datasets, using average search time, accuracy, and recall.

(3) Augmentation performance: experiments on WWk small compare downstream regression performance to verify the advantages of LLM-based intelligent augmentation over traditional methods.

(4) Response performance: end-to-end time is measured from the user's instruction to completion in ad hoc interactive scenarios. Results are averaged over multiple tests, with k = 10 by default.

(5) Stability testing: downstream performance trends are tested for different numbers of top-k tables to assess sensitivity to input changes and analyze their causes.

4.1.2 Test Configuration

Vectorization uses Jina Embedding v2 base-en, while MySQL and Pandas support storage and processing. Pandas, Qwen 2.5-max, and DeepSeek-V3 are integrated as augmentation components to improve data quality and representation. Missing-value processing uses scikit-learn and Google BERT (bert-base-uncased).

The contrastive module uses a backbone with six Transformer layers and eight attention heads. Training uses a batch size of 32, a learning rate of 5e-5, and a maximum token sequence length of 512. All experiments run on a computer with an NVIDIA RTX 3050 GPU and an Intel i5-12500H CPU. Because local computing resources are limited, the LLMs are accessed through remote APIs rather than deployed locally.

For testing, uvicorn starts the FastAPI service in the backend environment. The Vue application is then run in development mode using npm run dev in the frontend environment, completing deployment and startup.

Three datasets are used. Santos small is constructed from Canada's open data platform. Ten original tables are selected and processed into approximately 1,530 structured tables. The two WWk datasets are constructed from Wikipedia tables. Highly structured tables are selected, requiring at least ten rows and four valid columns to ensure sufficient information. For each query table, at least five highly related target tables are identified and labeled, ensuring sufficient related tables in every experiment.

WWk is divided into two subsets for different scales: WWk small contains approximately 550 tables, and WWk large approximately 3,500. These support evaluation of performance and accuracy in both small and large settings.

4.2 System Testing and Analysis

4.2.1 Functional Testing

Automated scripts simulate user operations across all modules and confirm correct operation. The system completes login/registration, related table discovery, augmentation, and downstream evaluation. All SDASDL modules produce expected results and support large-scale data lake augmentation tasks.

4.2.1.1 Login and Registration Module

As shown in Figure 4-1, users must log in or register before using SDASDL to enable identity verification and authentication. Email-based password recovery improves availability and user experience. Scripts simulate registration, login, and password recovery to verify correct request handling and responses.

Successful login after registration or password recovery confirms the module's correctness and usability.



(a) System login: account/email, password, log in, register, and forgot password.



(b) Account registration: email, password, confirm password, verification code, and register.



(c) Password recovery: email, verification code, new password, and reset password.

Figure 4-1. Login module interfaces (original application screenshots).

4.2.1.2 Table Augmentation Module

This module helps users augment tables and offers two ways to choose a query: upload a new table or select an earlier augmentation record. Figure 4-2 has three main areas: history on the left, the selected query table in the center, and an LLM conversation window on the right.

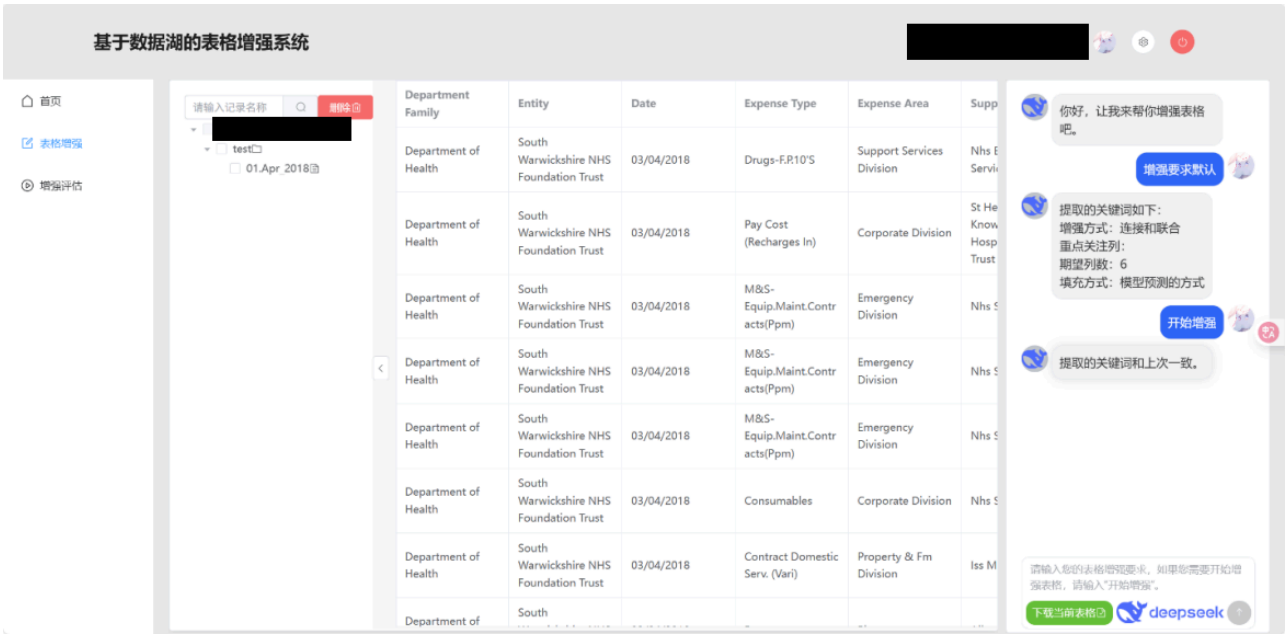


Figure 4-2. Table augmentation interface. Left: augmentation history. Center: query-table data. Right: conversation and augmentation requirements.

Users interact with the model through the conversation window. The model analyzes requirements and generates augmentation keywords. After confirming these keywords, the user enters "Start augmentation" to execute the subsequent operations. The system discovers related tables, performs augmentation, and immediately updates the central display with the result. This automates processing from requirement extraction to result presentation. Scripts based on the workflow in Figure 4-3 verify stability with different query-table sizes and correct exception handling.

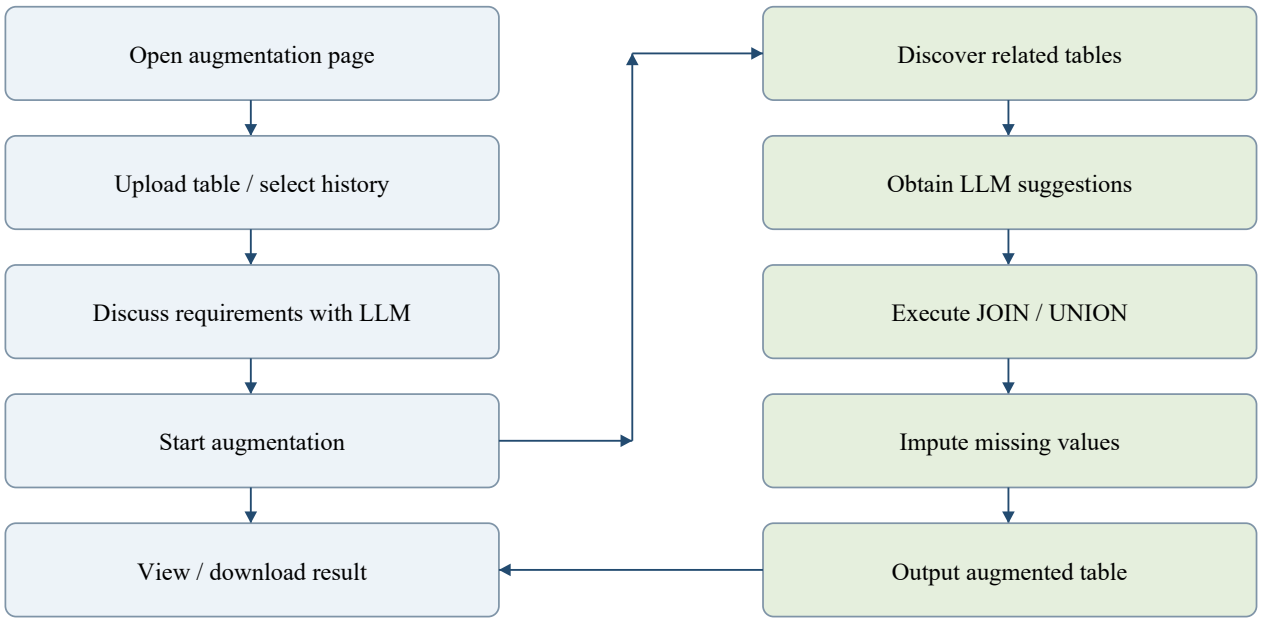


Figure 4-3. Table augmentation workflow.

4.2.1.3 Downstream Task Evaluation Page

The evaluation module has two core functions. First, its built-in XGBoost model supports prediction-task evaluation and quantitative analysis of augmentation. Second, an LLM code-writing feature generates customized code when default model parameters do not meet evaluation requirements, enabling more flexible configuration and comparisons.

Figure 4-4 shows the multi-table evaluation page. Users may upload up to five tables in the "Table upload" area. Uploaded tables appear in the "Uploaded files" list and can be deleted there. In "Training parameters", users enter model settings and select a common column as the prediction target. After "Start evaluation" is clicked, the system selects a metric automatically: RMSE for numerical data and F1-score for text data.

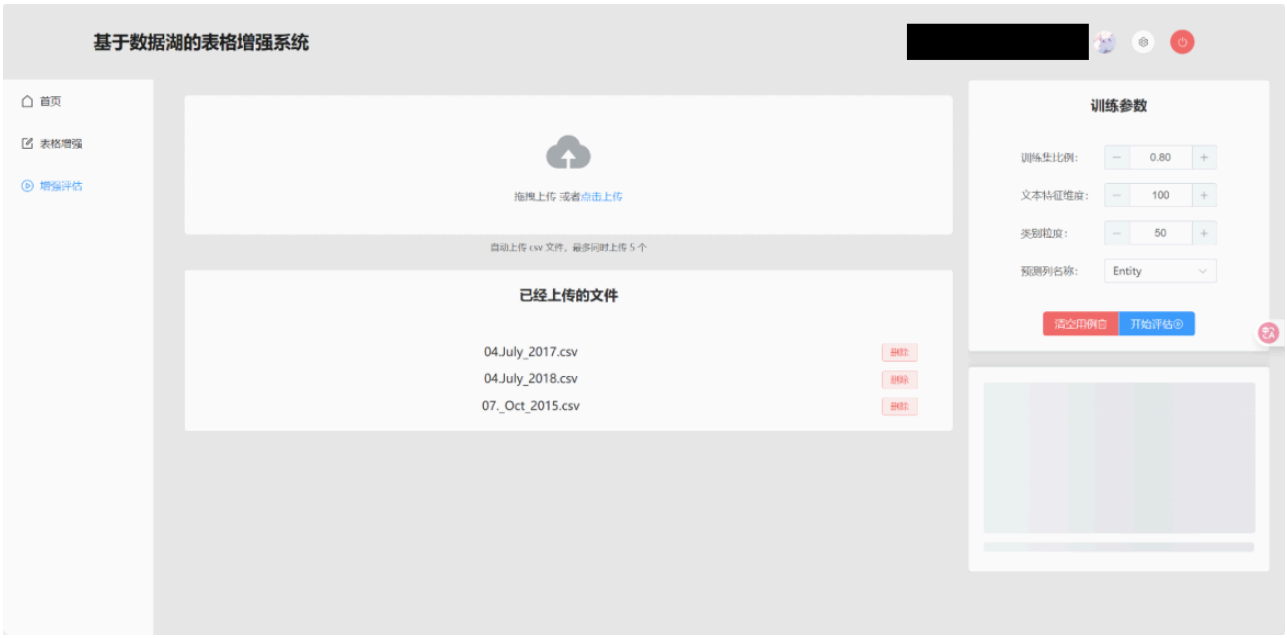


Figure 4-4. Multi-table prediction evaluation page. Controls include Table upload, Uploaded files, Training parameters, Target column, and Start evaluation.

After evaluation, the system generates a bar chart for each table's result, as shown in Figure 4-5, enabling straightforward comparisons.



Figure 4-5. Evaluation results page. The results dialog compares the prediction score for each uploaded table.

Automated scripts based on Figure 4-6 simulate adding and deleting tables and changing parameters. They verify that the module trains models and predicts correctly for different table types and sizes, returns results properly, and handles exceptions.

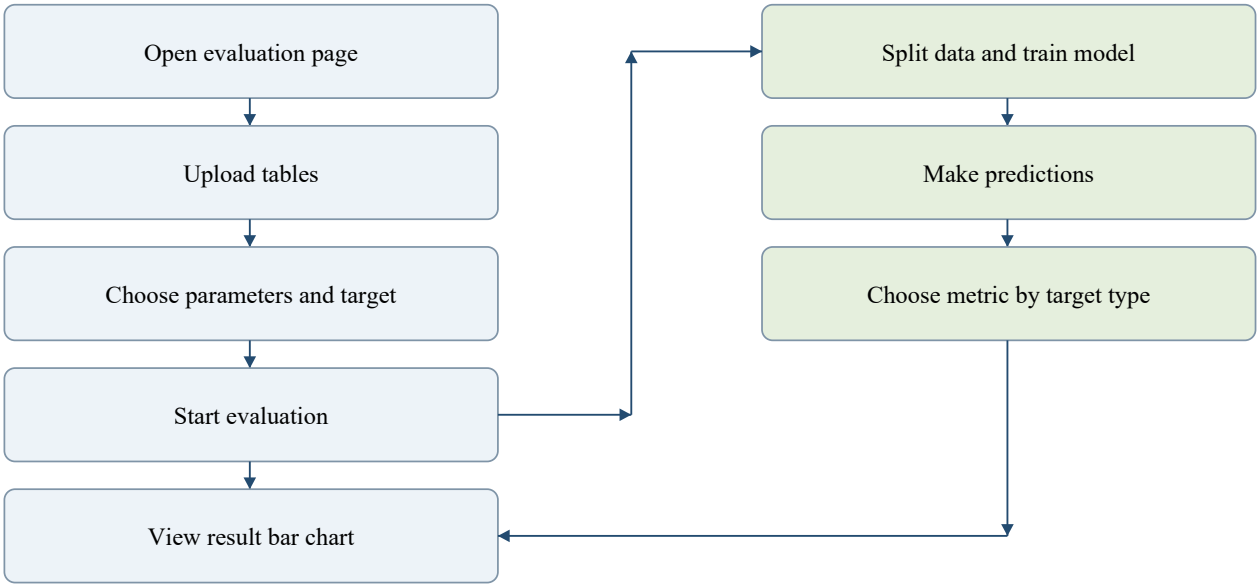


Figure 4-6. Downstream task evaluation workflow.

For additional requirements, an "AI-assisted writing" feature supports customization, as shown in Figure 4-7. Users enter existing code in the code box and specific writing requirements in the requirements area. After "Upload" is clicked, the system calls an LLM to generate code based on both inputs and displays the result in the code box, helping users write code quickly.

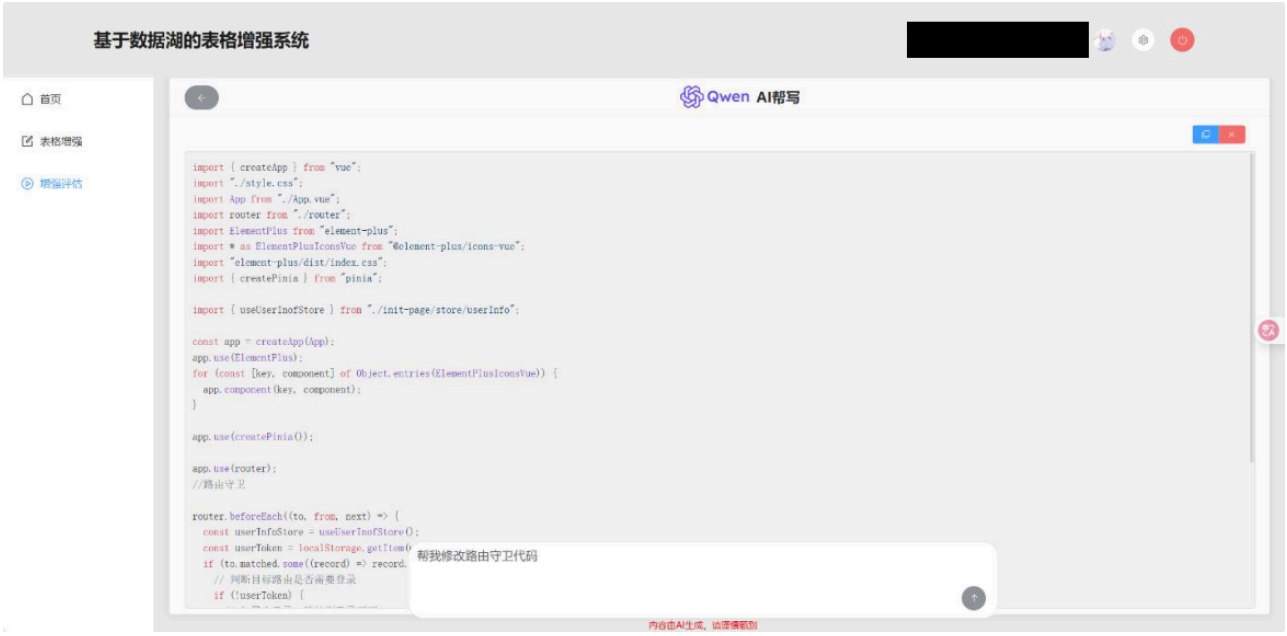


Figure 4-7. AI-assisted code-writing page. The main area contains the code editor and requirements input; Upload submits them to the LLM.

4.2.2 Related Table Discovery Performance

4.2.2.1 Baselines

The proposed HNSW-MST method is compared with the following baselines:

TF-IDF: a feature representation that weights local term frequency by inverse document frequency across the corpus to measure table similarity.

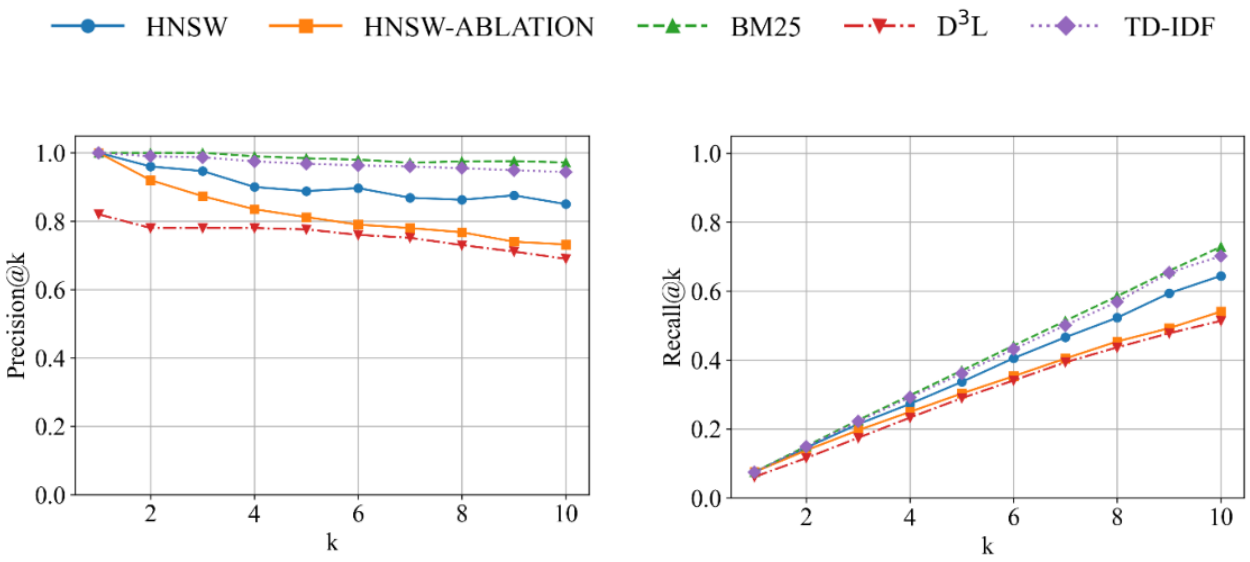
BM25: a probabilistic retrieval scoring function that introduces term-frequency saturation and column-length normalization to improve similarity assessment.

D3L: a table union-search method using multiple dimensions, including column names, value overlap, and format similarity.

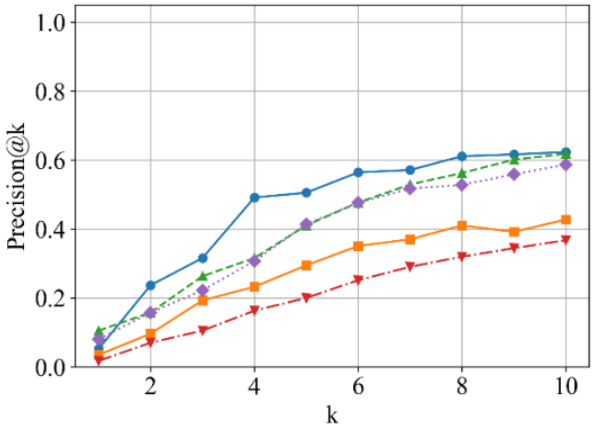
The comparisons use accuracy and recall, together with average search speed on different datasets, to verify advantages in both correctness and efficiency.

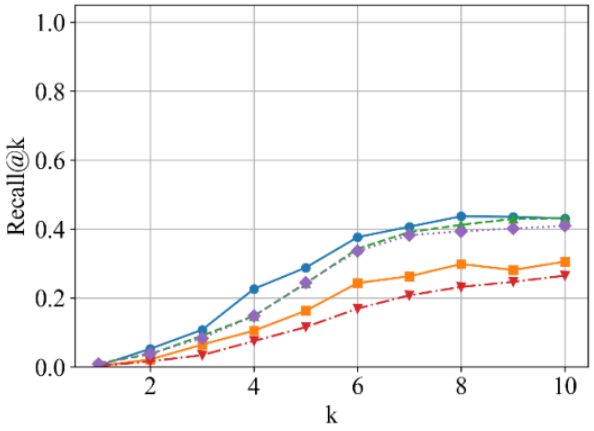
4.2.2.2 Results

Figure 4-8 compares discovery accuracy and recall for the system and baselines on three datasets.

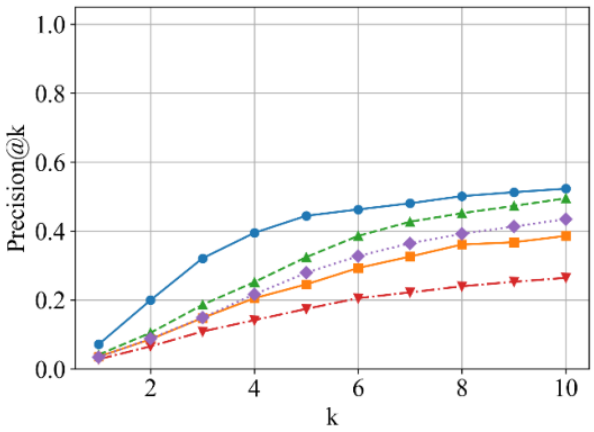


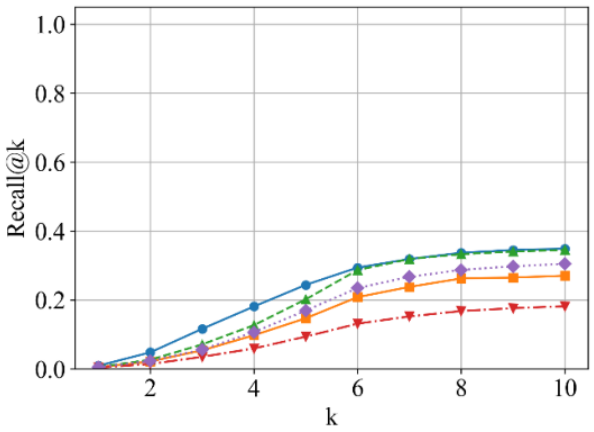
(a) P@k on Santos small. (b) R@k on Santos small.





(c) P@k on WWk small. (d) R@k on WWk small.





(e) P@k on WWk large. (f) R@k on WWk large.

Figure 4-8. Comparison of related table discovery precision and recall.

Figure 4-8 shows that, on WWk, the HNSW-based method improves accuracy and recall by approximately 10% over the strongest baseline, BM25. On Santos, it performs slightly worse than BM25 and TF-IDF. This difference may result from the simpler construction of Santos: large tables are randomly split into smaller ones, creating a high proportion of repeated fields between related tables. Text-matching methods therefore have an advantage. WWk tables, by contrast, originate from Wikipedia pages and are primarily related through semantics rather than textual matches. The proposed method more effectively captures semantics within and between tables and performs better on this dataset.

Ablation experiments test the importance of intercolumn modeling by comparing the full HNSW method with HNSW-ABLATION, which removes the column-semantic Transformer. The full model performs better on all three datasets.

Average improvements in accuracy and recall exceed 20%, demonstrating that column-semantic modeling substantially improves discovery accuracy.

The proposed retrieval method also outperforms all baselines in query latency, reducing average latency by more than 50% on every dataset. Table 4-1 provides the values.

Table 4-1. Average query latency (seconds).

Method	Santos small	WWk small	WWk large
HNSW	0.0213	0.0157	0.1492
D3L	0.7800	0.2959	0.6137
BM25	0.3055	0.0360	0.3301
TF-IDF	0.3515	0.0757	0.5782

Table 4-1 shows that the method is faster than all baselines. On Santos, it is 90% faster than the fastest baseline; on WWk, latency is reduced by more than 50%. The method therefore has clear speed advantages at different scales. Analysis of table sizes indicates that Santos tables are larger on average, increasing latency for text-based BM25 and TF-IDF. Smaller WWk tables allow these baselines to run relatively faster.

4.2.3 Table Augmentation Performance

4.2.3.1 Baselines

The LLM-based intelligent augmentation method is compared with four approaches:

(1) Simple baseline join: directly joins same-named columns of a randomly selected related table; this is the most basic strategy.

(2) Statistical baseline join: uses intercolumn similarity to choose the most compatible columns.

(3) Random filtered join: excludes columns with excessively high or low unique-value ratios, then randomly selects a key for a left join.

(4) Column-name matching baseline join: determines join columns using string similarity between names.

The methods are compared on downstream regression using mean squared error (MSE), root mean squared error (RMSE), mean absolute error (MAE), and the coefficient of determination (R^2).

Comparing these metrics demonstrates the effectiveness of the proposed method in improving downstream prediction.

4.2.3.2 Results

Table 4-2 reports average improvements in downstream regression metrics for tables augmented using different methods on WWk small.

Table 4-2. Improvement in prediction metrics (%).

Method	MSE	RMSE	MAE	R^2
LLM-based intelligent augmentation	46.15	46.15	46.15	65.38
Simple baseline join	19.23	19.23	23.07	15.38
Statistical baseline join	3.84	3.84	3.84	3.84
Random filtered join	19.23	19.23	19.23	23.07
Column-name matching baseline	7.69	7.69	7.69	7.69

LLM-based intelligent augmentation outperforms the other baselines. MSE, RMSE, and MAE each improve by 46.15%, while R^2 improves by 65.38%, substantially exceeding the second-best simple baseline join. Statistical and column-name matching baselines provide limited improvements, indicating that strategies relying only on names or unique-value ratios are ineffective. By understanding semantic relationships more deeply through an LLM, the proposed method markedly improves prediction performance and demonstrates its augmentation advantages.

4.2.4 Response Performance

To assess practical responsiveness, Figure 4-9 shows total and per-module time as the number of tables to augment increases. Discovery uses the proposed HNSW-like method, augmentation uses top-10 related tables, and the tested table count increases from 50 to 500.

Total time grows approximately linearly from 15 to 155 seconds, averaging about 0.31 seconds per table. The total-time curve approximately overlaps the augmentation-time curve, indicating that augmentation is the main bottleneck. Discovery takes hundredths of a second and is negligible. In practical interactive use, augmenting one table takes less than ten seconds, meeting usability requirements. SDASDL therefore provides acceptable end-to-end latency and usability in its current implementation.

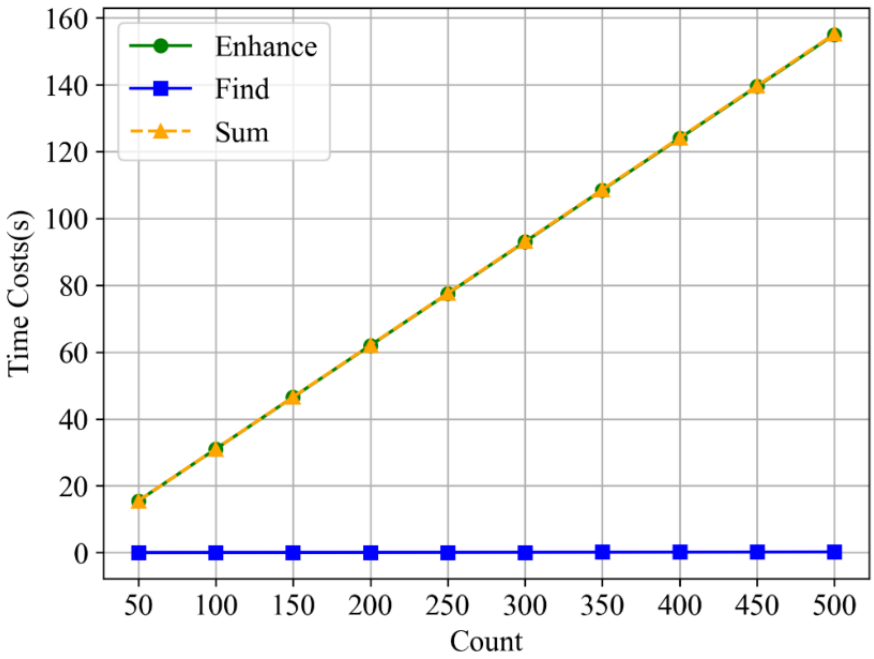
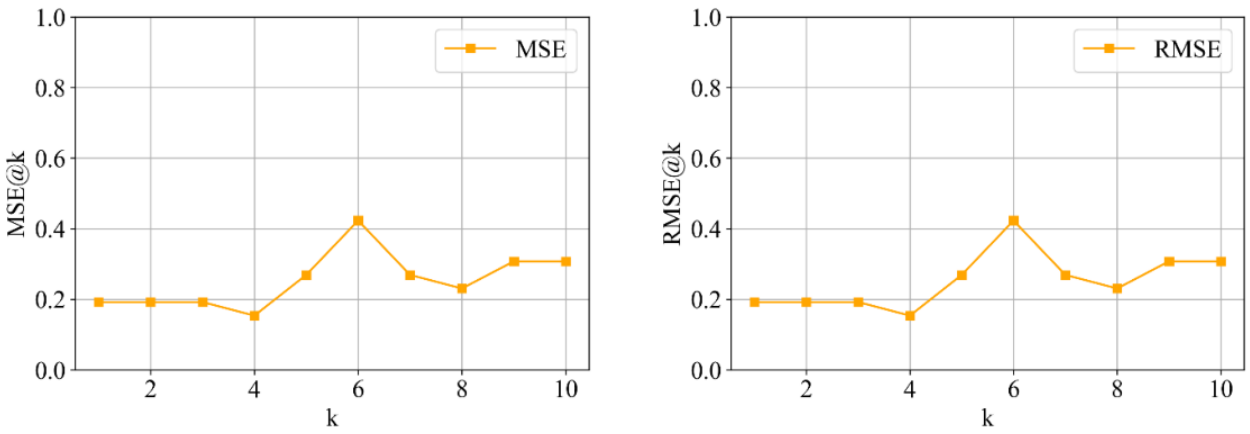


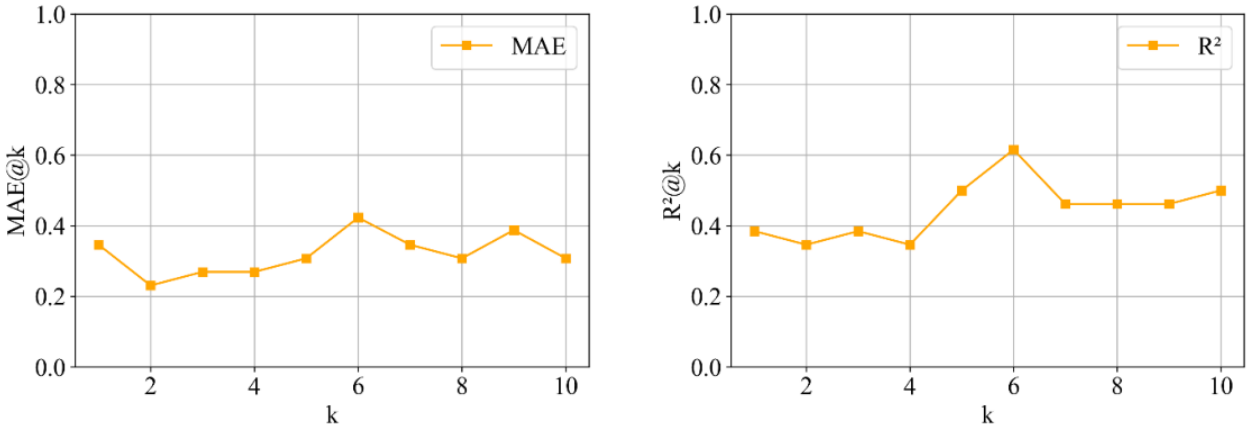
Figure 4-9. SDASDL processing-time curves for different numbers of tables.

4.2.5 Stability Testing

Following prior work evaluating the influence of the related-table count k [1]-[14], this thesis tests four downstream machine learning metrics as the number of candidate tables increases from 1 to 10. Figure 4-10 shows the results.



(a) MSE. (b) RMSE.



(c) MAE. (d) R².

Figure 4-10. Augmentation performance with different numbers of related tables.

Overall, the curves show slight early fluctuations, followed by an increase and then a decrease toward stability. Performance peaks at k = 6. When k is at most 4, all four metrics are low and fluctuate slightly, indicating insufficient related-table information and weak augmentation. At k = 6, the system performs best: additional information allows the LLM to provide optimal suggestions. As k increases further to 10, excessive redundant information makes inputs too long and impairs LLM decisions, reducing performance.

The results show a clear turning point in sensitivity to k. A small number of tables (k at most 4) provides limited information and only basic improvement. Increasing k improves performance until the peak at 6. For k at least 7, redundancy interferes with decision-making. Accordingly, k = 6 is selected for practical use.

4.3 Chapter Summary

This chapter evaluates discovery and augmentation through experiments across several dimensions. The proposed method has clear advantages in semantically complex settings: on WWk, accuracy and recall improve by about 10% over BM25, and query latency falls by more than 60%. Ablation experiments show that the column-level Transformer improves discovery accuracy by over 20%, confirming the importance of intercolumn semantics. In downstream regression, LLM-based augmentation improves all four metrics by more than 40%, substantially outperforming traditional column-name matching. These results demonstrate overall advantages in accuracy, efficiency, and applicability.

The chapter also systematically presents SDASDL's design and implementation, showing that efficient querying and appropriate use of LLMs enable automated, intelligent augmentation.

Its layered architecture and frontend/backend separation support scalability and operational efficiency. The system includes login/registration, discovery, augmentation, and downstream evaluation modules. Multiple tests verify functional completeness, usability, and accuracy throughout the end-to-end workflow, demonstrating the method's effectiveness and practicality. The results show that SDASDL improves downstream machine learning performance while maintaining system stability, supporting its feasibility in real data lake environments.

Chapter 5 Conclusions and Future Work

5.1 Summary of the Work

As data lake technologies develop rapidly, structured data volumes and table counts continue to grow. Efficiently using these data has become an urgent problem. Traditional augmentation struggles to capture semantic relationships between columns and incurs substantial overhead at large scales. This thesis therefore proposes and implements a semantically driven system for efficient, automated augmentation of structured data lake tables. Its main contributions are as follows.

(1) A complete automated system is designed and implemented, covering preprocessing, discovery, augmentation-strategy generation and execution, and imputation, with strong generality and scalability.

(2) An efficient TF-IDF-based vectorization method converts tables of different sizes into fixed-length representations, compressing storage and substantially improving vectorization efficiency while preserving semantics.

(3) An HNSW graph index is constructed using custom table similarity. A fast discovery strategy combining simulated annealing supports low-latency retrieval for user queries.

(4) An LLM-based suggestion-generation method and a structured suggestion format enable more intelligent augmentation and improve result quality and usability.

(5) Experiments on several real datasets systematically compare discovery and augmentation methods. The proposed approach offers better discovery speed and accuracy, while augmented tables improve downstream prediction performance by more than 40%, verifying effectiveness and practicality.

5.2 Future Work

The system has largely achieved its design goal of end-to-end augmentation. Discovery accuracy, processing speed, and latency have reached practical levels. Some limitations still require further investigation.

(1) Improving the definition of table similarity

The current definition effectively captures structural and semantic relationships, particularly intercolumn semantics. However, it produces a numerical similarity score, making direct use of efficient vector indexes for fast queries difficult.

This affects retrieval efficiency over large table collections. Future work could encode each entire table into a single vector without losing semantic expressiveness, preserving meaningful representation while improving retrieval efficiency.

(2) Optimizing overall latency

Although augmentation of one table is currently completed within 60 seconds and meets basic requirements, limited local resources and remote LLM API calls constrain concurrency. Local LLM deployment and more powerful hardware could improve throughput and support highly concurrent augmentation.

(3) Simplifying and standardizing augmentation suggestions

The current format is structured and interpretable and effectively represents the core elements of operations. However, complex scenarios leave room to improve parsing convenience. LLM suggestions may also contain errors such as inconsistent column names, affecting accuracy. Future work should optimize their representation using a more structured, standardized, easily parsed format to improve stability and automation.

References

[1] Khatiwada A, Fan G, Shraga R, et al. Santos: Relationship-Based Semantic Table Union Search[J]. Proceedings of the ACM on Management of Data, 2023, 1(1): 1-25.

[2] Bogatu A, Fernandes A A A, Paton N W, et al. Dataset Discovery in Data Lakes[C]. 2020 IEEE 36th International Conference on Data Engineering (ICDE). IEEE, 2020: 709-720.

[3] Nargesian F, Zhu E, Pu K Q, et al. Table Union Search on Open Data[J]. Proceedings of the VLDB Endowment, 2018, 11(7): 813-825.

[4] Chen P B, Zhang Y, Roth D. Is Table Retrieval a Solved Problem? Exploring Join-Aware Multi-Table Retrieval[C]. Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). 2024: 2687-2699.

[5] Zhu E, Deng D, Nargesian F, et al. Josie: Overlap Set Similarity Search for Finding Joinable Tables in Data Lakes[C]. Proceedings of the 2019 International Conference on Management of Data. 2019: 847-864.

[6] Harmouch H, Papenbrock T, Naumann F. Relational Header Discovery Using Similarity Search in a Table Corpus[C]. 2021 IEEE 37th International Conference on Data Engineering (ICDE). IEEE, 2021: 444-455.

[7] Santos A, Bessa A, Musco C, et al. A Sketch-Based Index for Correlated Dataset Search[C]. 2022 IEEE 38th International Conference on Data Engineering (ICDE). IEEE, 2022: 2928-2941.

[8] Dong Y, Takeoka K, Xiao C, et al. Efficient Joinable Table Discovery in Data Lakes: A High-Dimensional Similarity-Based Approach[C]. 2021 IEEE 37th International Conference on Data Engineering (ICDE). IEEE, 2021: 456-467.

[9] Esmailoghli M, Quiané-Ruiz J A, Abedjan Z. MATE: Multi-Attribute Table Extraction[J]. Proceedings of the VLDB Endowment, 2022, 15(8): 1684-1696.

[10] Chepurko N, Marcus R, Zgraggen E, et al. ARDA: Automatic Relational Data Augmentation for Machine Learning[J]. Proceedings of the VLDB Endowment, 2020, 13(9).

[11] Zhao Z, Castro Fernandez R. Leva: Boosting Machine Learning Performance with Relational Embedding Data Augmentation[C]. Proceedings of the 2022 International Conference on Management of Data. 2022: 1504-1517.

[12] Castelo S, Rampin R, Santos A, et al. Auctus: A Dataset Search Engine for Data Discovery and Augmentation[J]. Proceedings of the VLDB Endowment, 2021, 14(12): 2791-2794.

[13] Khatiwada A, Shraga R, Miller R J. DIALITE: Discover, Align and Integrate Open Data Tables[C]. ACM SIGMOD. 2023, 1(1).

[14] Fan G, Wang J, Li Y, et al. Semantics-Aware Dataset Discovery from Data Lakes with Contextualized Column-Based Representation Learning[J]. Proceedings of the VLDB Endowment, 2023, 16(7).

[15] Stekhoven D J, Bühlmann P. MissForest-Non-Parametric Missing Value Imputation for Mixed-Type Data[J]. Bioinformatics, 2012, 28(1): 112-118.

[16] Yoon J, Jordon J, Schaar M. Gain: Missing Data Imputation Using Generative Adversarial Nets[C]//International Conference on Machine Learning. PMLR, 2018: 5689-5698.

[17] Zhu E, He Y, Chaudhuri S. Auto-Join: Joining Tables by Leveraging Transformations[J]. Proceedings of the VLDB Endowment, 2017, 10(10): 1034-1045.

[18] Devlin J, Chang M W, Lee K, et al. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding[C]. Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies. 2019: 4171-4186.

[19] Vaswani A, Shazeer N, Parmar N, et al. Attention is All You Need[C]. Advances in Neural Information Processing Systems. 2017, 30.

[20] Chen T, Kornblith S, Norouzi M, et al. A Simple Framework for Contrastive Learning of Visual Representations[C]. Proceedings of the 37th International Conference on Machine Learning. 2020: 1597- 1607.

[21] Günther M, Ong J, Mohr I, et al. JINA EMBEDDINGS2: 8192-Token General-Purpose Text Embeddings for Long Documents[EB/OL]. arXiv preprint arXiv:2310.19923, 2023.

[22] Sinkhorn R. A Relationship between Arbitrary Positive Matrices and Doubly Stochastic Matrices[J]. The Annals of Mathematical Statistics, 1964, 35(2): 876-879.